TRAKYA ÜNİVERSİTESİ · BİLGİSAYAR MÜHENDİSLİĞİ

Dosya Organizasyonu

Hafta 4 · Kayıt Biçimleri ve Hashing

Doç. Dr. Emir Öztürk

2025/2026 Güz Yarıyılı

1

Field (alan) ve record (kayıt) organizasyonu

Sınırların tanınması için kullanılan yöntemler

SORUN

Stream file (akım dosyası)



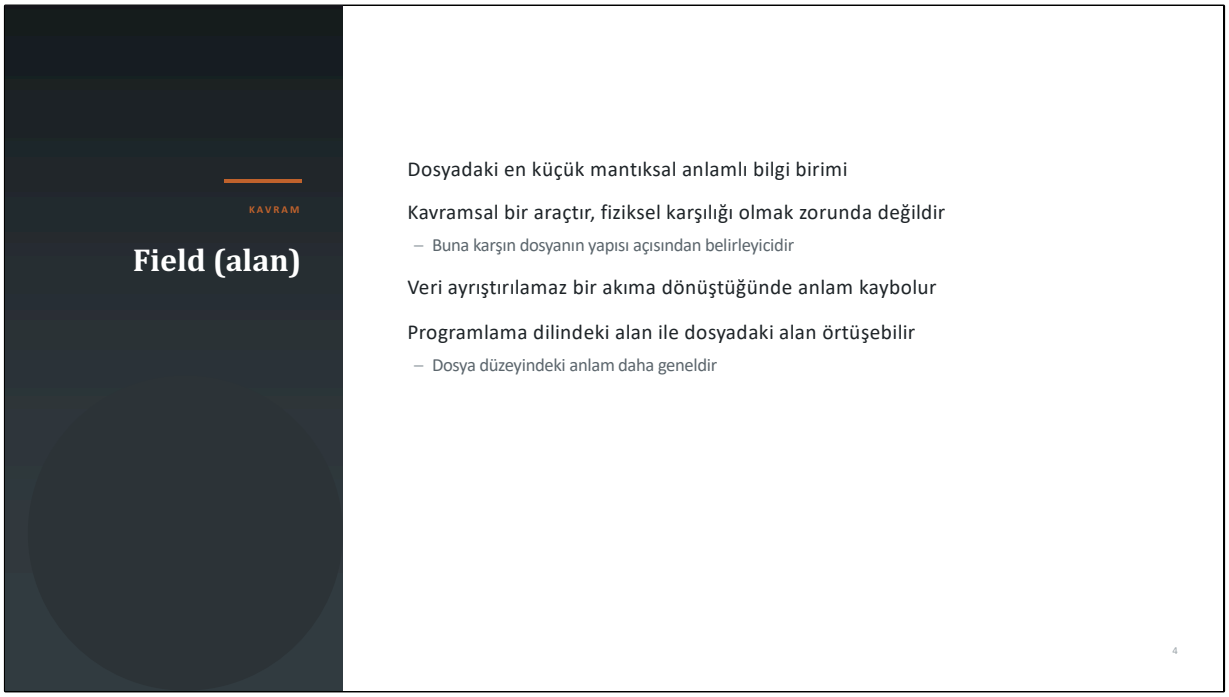
Alan sınırları geri getirilemez

Veri file (dosyaya) olduğu gibi yazılır, ek bilgi konmaz

Bilgi eksiksiz taşınır, ama geri ayrıştırılamaz

Dosya yapısı tasarımının ilk gerekçesi budur

Verinin file (dosyaya) olduğu gibi, hiçbir ek bilgi konmadan yazılması durumunda ortaya bir bayt akımı çıkar. Akım, istenen bilgiyi eksiksiz taşır. Ancak bilginin hangi bölümünün hangi field (alana) ait olduğu geri getirilemez. Ad, adres ve şehir bilgileri birbirine yapışır ve ayrıştırılamaz duruma gelir. Bu durum, dosya yapısı tasarımının ilk gerekçesidir: veriyi yazmak yeterli değildir, veriyi geri okunabilir kılan bir düzen de gereklidir.



Field (alan), bir dosyadaki en küçük mantıksal anlamlı bilgi birimi olarak tanımlanır. Alan kavramsal bir araçtır. Dosyada fiziksel bir karşılığı bulunmak zorunda değildir. Buna karşılık dosyanın yapısı açısından belirleyicidir, çünkü verinin anlamı ancak alanlara bölünmüş olduğunda korunur. Programlama dillerindeki alan ve record (kayıt) terimleriyle dosya düzeyindeki karşılıkları arasında sık sık örtüşme bulunur, ancak dosya düzeyindeki anlamları daha geneldir.

YÖNTEM

Field (alanların) düzenlenmesi · dört yol

(a) Sabit uzunluk

```
Ames      Mary      123 Maple      Stillwater  OK74075
```

(b) Uzunluk göstergesi

```
04Ames04Mary09123 Maple10Stillwater02OK0574075
```

(c) Ayraç

```
Ames|Mary|123 Maple|Stillwater|OK|74075|
```

(d) Anahtar = değer

```
last=Ames|first=Mary|address=123 Maple|city=Stillwater|
```

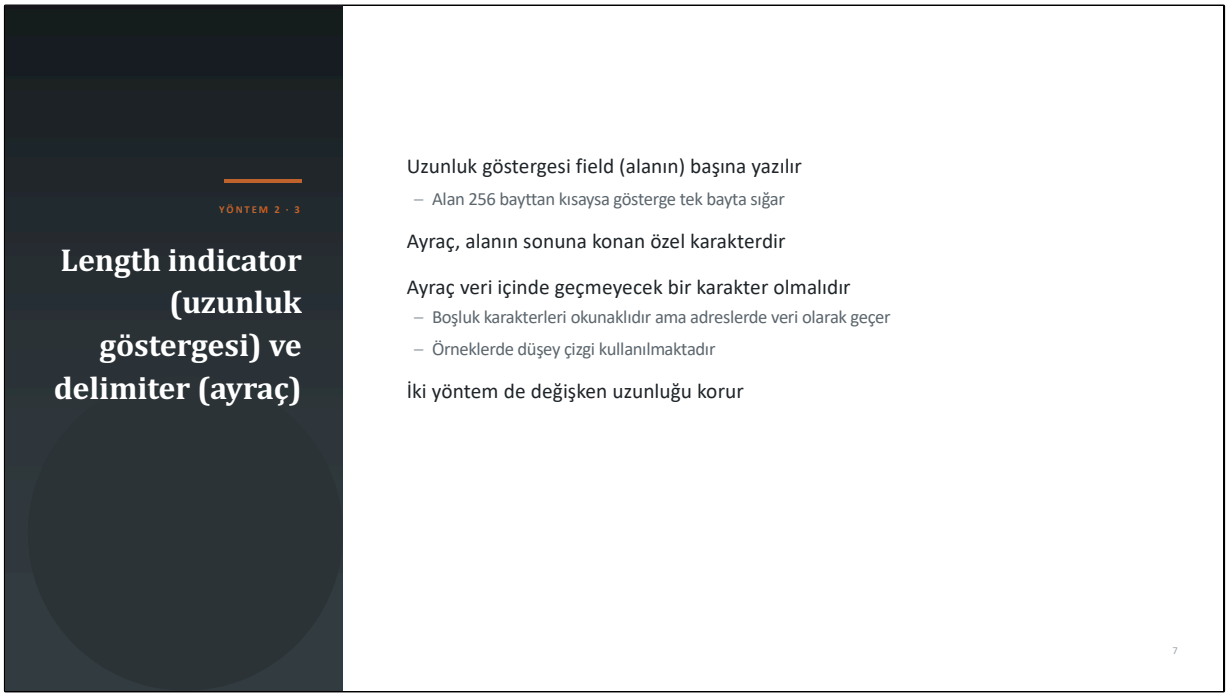
Seçim, alan uzunluklarındaki değişkenliğe göre yapılır

5

Field (alanların) tanınması için dört yöntem kullanılmaktadır. Birinci yöntemde alanlar öngörülebilir uzunluğa zorlanır ve sınırlar sayımla bulunur. İkinci yöntemde her alanın başına uzunluğunu bildiren bir gösterge konur. Üçüncü yöntemde alanlar, veride geçmeyeceği bilinen bir delimiter (ayraç) karakteriyle ayrılır. Dördüncü yöntemde her alan kendi adıyla birlikte yazılır. Yöntemlerin seçimi verinin değişkenliğine ve okuma biçimine göre yapılır.



Fixed-length field (sabit uzunluklu alan) yaklaşımında her field (alana), taşıyabileceği en uzun değere yetecek yer ayrılır. Sınırların bulunması yalnızca toplama işlemi gerektirdiği için okuma basittir. Buna karşılık iki sorun ortaya çıkar. Kısa değerler için ayrılan yerin büyük bölümü boş kalır. Tanımlanan sınırdan uzun değerler ise dosyaya sığmaz. İkinci sorunu çözmek için sınırların büyütülmesi birinci sorunu ağırlaştırır. Yöntem, alan uzunlukları arasında az değişkenlik bulunan veriler için uygundur.



Length indicator (uzunluk göstergesi) yönteminde field (alanın) başına, alanın kaç bayt olduğunu bildiren bir sayı yazılır. Alan uzunlukları 256 baytın altında kaldığında bu sayı tek bir bayta sığar. Delimiter (ayraç) yönteminde ise alanın sonuna, veri içinde geçmeyeceği bilinen özel bir karakter konur. Ayraç seçimi önem taşır: boşluk karakterleri konsol çıktısında okunaklı bir ayrım sağlasa da adres alanlarında veri olarak geçtiğinden kullanılamaz.

YÖNTEM 4

Key (anahtar) = değer

Her field (alan) kendi adıyla birlikte yazılır

Alan kendi kendini tanımlar

- Dosyanın hangi alanları taşıdığı önceden bilinmeden okunur
- Eksik alan sorun çıkarmaz: adı dosyada bulunmaz

Bedeli yer kullanımıdır

- Kısa alanlarda adlar dosyanın yarısından çoğunu kaplayabilir

Veri hacmi ad uzunluğuna göre büyükse anlamlıdır

Dördüncü yöntemde her field (alan), adı ve değeri birlikte yazılarak kendi kendini tanımlar. Bunun iki üstünlüğü vardır: dosyanın hangi alanları taşıdığı önceden bilinmeden okunabilir ve eksik alanlar sorun çıkarmaz, çünkü eksik alanın adı dosyada bulunmaz. Bedeli yer kullanımıdır. Kısa alanlardan oluşan bir dosyada alan adları toplam yerin yarısından çoğunu kaplayabilir. Yöntem, veri hacminin ad uzunluğuna göre büyük olduğu biçimlerde anlamlı duruma gelir.

KAVRAM

Record (kayıt)

Birlikte anlam taşıyan field (alanlar) kümesi

Alan gibi kayıt da mantıksal bir birimdir

Memory (bellekteki) karşılığı nesne, dosyadaki karşılığı kayıttır

- Yazma: nesnenin durumunun saklanması
- Okuma: nesnenin durumunun geri kurulması

Dosya yapısı tasarımının amacı bu taşımayı kolaylaştırmaktır

Record (kayıt), dosya daha üst bir örgütlenme düzeyinde görüldüğünde birlikte anlam taşıyan field (alanlar) kümesi olarak tanımlanır. Alan gibi kayıt da mantıksal bir birimdir. Memory (bellekteki) nesnenin dosyadaki karşılığı kayıt, kaydın bellekteki karşılığı nesnedir. Dosya yapısı tasarımının amacı bu iki temsil arasındaki taşımayı kolaylaştırmaktır. Nesnenin dosyaya yazılması, o nesnenin durumunun saklanması, dosyadan okunması ise durumunun geri kurulması anlamına gelir.

YÖNTEM

Record (kayıtların) tanınması · beş yol

Kaydın bayt sayısının sabitlenmesi

Kaydın field (alan) sayısının sabitlenmesi

Kaydın başına length indicator (uzunluk göstergesi) konması

Adreslerin ayrı bir indeks dosyasında tutulması

Kaydın sonuna delimiter (ayraç) konması

Uygulamada birden çok yöntem birlikte kullanılır

- Fixed-length record (sabit uzunluklu kaydın) içine variable-length field (değişken uzunluklu alanlar) konabilir

Record (kayıtların) tanınması için beş yöntem kullanılır: kaydın bayt sayısının sabitlenmesi, field (alan) sayısının sabitlenmesi, kaydın başına length indicator (uzunluk göstergesi) konması, adreslerin ayrı bir indeks dosyasında tutulması ve kaydın sonuna delimiter (ayraç) konması. Bu yöntemler alanlar için sayılan yöntemlerin bir üst düzeydeki karşılığıdır. Uygulamada birden çok yöntem birlikte kullanılır. Sabit uzunluklu bir kaydın içine variable-length field (değişken uzunluklu alanlar) yerleştirilmesi bunun yaygın bir örneğidir.

YAPI

Fixed-length record (sabit uzunluklu kayıt)



Record (kaydın) yeri çarpmayla bulunur: direct access (doğrudan erişim) olanaklı

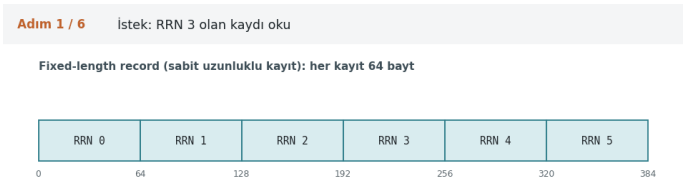
Bayt sayısının sabitlenmesi field (alan) sayısını sabitlemez

11

Fixed-length record (sabit uzunluklu kayıt), her record (kaydın) aynı sayıda bayt taşıdığı düzendir. Bu düzenin belirleyici üstünlüğü, herhangi bir kaydın başlangıç konumunun çarpma işlemiyle hesaplanabilmesi ve böylece direct access (doğrudan erişimin) olanaklı duruma gelmesidir. Kaydın bayt sayısının sabitlenmesi, field (alan) sayısının ya da alan uzunluklarının da sabit olmasını gerektirmez. Sabit uzunluklu kayıtlar, variable-length field (değişken uzunluklu alanlar) için standart boyda bir kap olarak sıkça kullanılır.

HAREKETLI ŞEKIL

Direct access (doğrudan erişim)



Konum = RRN × kayıt boyu: tek seek, tek okuma

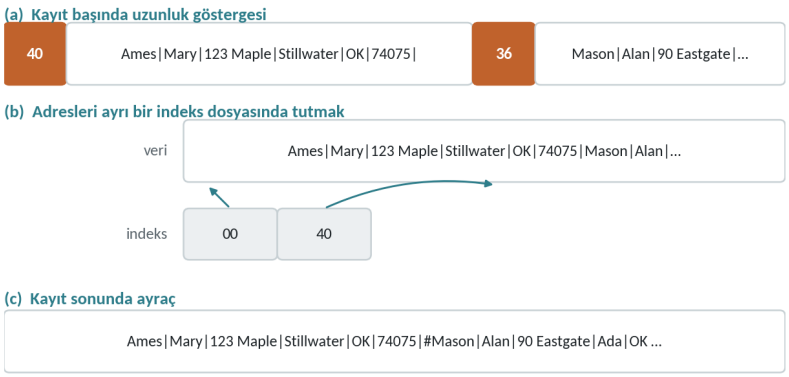
Değişken kayıtta önceki kayıtlar tek tek atlanır

12

Animasyon altı adımdan oluşmakta ve her adım yaklaşık üç saniye ekranda kalmaktadır. Her biri altmış dört bayt olan altı kayıtlık bir dosyada üç numaralı kaydın okunması istenmektedir. Kaydın bayt konumu göreli kayıt numarasının kayıt boyuyla çarpılmasıyla, 3 × 64 = 192 olarak bulunmakta ve okuma göstericisi öteki kayıtlara hiç dokunmadan doğrudan 192. bayta konumlandırılmaktadır. Ardından altmış dört bayt tek seferde okunmaktadır: bir seek ve bir okuma. Son iki adım aynı isteği variable-length record (değişken uzunluklu kayıt) içeren bir dosyada göstermektedir. Kayıtların boyu farklı olduğundan üçüncü kaydın yeri çarpmayla bulunamaz; sıfırıncı, birinci ve ikinci kayıtların uzunluk alanları sırayla okunup atlanmak zorundadır. Hash adresinin bir RRN olması ve direct access (doğrudan erişimli) dosyaların sabit uzunluklu kayıtlarla kurulması bu farktan kaynaklanır.

YAPI

Variable-length record (değişken uzunluklu kayıt)



Length indicator (uzunluk göstergesi) en yaygın kullanılan yöntemdir

13

Variable-length record (değişken uzunluklu kayıtlar) için üç yapı ele alınmaktadır. Length indicator (uzunluk göstergesi) yönteminde record (kaydın) başındaki sayı, kaydın geri kalanının kaç bayt olduğunu bildirir. Bu yöntem değişken uzunluklu kayıtların işlenmesinde en yaygın kullanılandır. İndeks dosyası yönteminde her kaydın başlangıç uzaklığı ikinci bir dosyada tutulur ve kayıt uzunlukları ardışık uzaklıkların farkından hesaplanır. Delimiter (ayraç) yönteminde kaydın sonuna özel bir karakter konur. Satır sonu karakteri bu amaçla sık kullanılır.

İŞLEM

Değişken record (kaydın) yazılması

Uzunluk kaydın başına yazılacaksa önce bilinmelidir

- Field (alanlar) önce bellekte bir buffer (tamponda) birleştirilir
- Uzunluk tampon üzerinden hesaplanır
- Önce uzunluk, sonra tamponun içeriği yazılır

Uzunluk iki biçimde yazılabilir

- Sabit boyutlu ikili tamsayı: az yer, büyük değer aralığı
- Karakter dizisi: dosya ekranda okunabilir kalır

Hiçbiri tek doğru değildir, ortama göre seçilir

Length indicator (uzunluk göstergesinin) record (kaydın) başına yazılabilmesi için kaydın toplam uzunluğunun yazma işleminden önce bilinmesi gerekir. Bu nedenle field (alanlar) önce bellekte bir buffer (tamponda) birleştirilir, uzunluk tampon üzerinden hesaplanır ve ardından önce uzunluk, sonra tamponun içeriği dosyaya yazılır. Uzunluğun kendisi iki biçimde yazılabilir: sabit boyutlu ikili tamsayı olarak ya da ondalık basamakların karakter karşılığı olarak. İkili biçim daha az yer kaplar ve daha büyük değerleri taşır. Karakter biçimi ise dosyayı ekranda okunabilir kılar.

HAREKETLI ŞEKIL

Değişken record (kaydın) yazılması



Alanlar önce buffer (tamponda) birleşir; uzunluk ancak o zaman bilinir

Dosyaya önce uzunluk, ardından tamponun içeriği yazılır

15

Animasyon on bir adımdan oluşmakta ve her adım yaklaşık üç saniye ekranda kalmaktadır. Bellekteki nesnenin altı alanı sırayla tampona eklenmekte ve her alanın ardından ayraç olarak dikey çizgi konmaktadır. Sağ üstteki sayaç tampondaki bayt sayısını göstermektedir. Altı alan eklendiğinde kaydın uzunluğu kırk bayt olarak bilinir duruma gelmektedir. Önce bu uzunluk iki baytlık bir tamsayı olarak dosyaya yazılmakta, ardından tamponun kırk baytı tek seferde yazılmaktadır. Son adımda ikinci kayıt aynı yolla eklenmekte, otuz altı baytlık içerik kırk ikinci bayttan başlayarak dosyaya geçmektedir. Uzunluğun kaydın başına yazılabilmesi için kaydın önce bellekte tamamlanması gerektiği bu sırada açıkça görülmektedir.

İŞLEM

Değişken record (kaydın) okunması

Önce uzunluk field (alanı) okunur

Uzunluk kadar bayt tek erişimde tampona alınır

Tampon, alanlarına ayrıştırılır

- Dosyadan okuma için yazılmış işlevler tampon üzerinde yeniden kullanılır
- C++ dizge akımları bunu olanaklı kılar

Tek erişimle okuma, erişim sayısını da azaltır

Değişken uzunluklu bir record (kaydın) okunması üç adımdan oluşur. Önce uzunluk field (alanı) okunur, sonra bu uzunluk kadar bayt tampona alınır, en sonunda tampon alanlarına ayrıştırılır. Ayrıştırma, dosyadan doğrudan okuma için yazılmış işlevlerin tampon üzerinde yeniden kullanılmasıyla yapılabilir. C++ dizge akımları bunu olanaklı kılar. Bu yaklaşım, kaydın dosyadan tek bir erişimle alınmasını sağladığı için erişim sayısını da azaltır.

HAREKETLI ŞEKIL

Değişken record (kaydın) okunması



Uzunluk alanı, kaç bayt okunacağını söyler

Kayıt tek erişimde tampona gelir, alanlar orada ayrılır

17

Animasyon sekiz adımdan oluşmakta ve her adım yaklaşık üç saniye ekranda kalmaktadır. Okuma göstericisi dosyanın başındadır ve kaydın boyu henüz bilinmemektedir. Önce iki baytlık uzunluk alanı okunmakta ve kaydın kırk bayt olduğu öğrenilmektedir. Ardından kırk bayt tek erişimde tampona alınmakta, gösterici kırk ikinci bayta ilerlemektedir. Tampon dikey çizgilerden bölünerek alanlar nesneye aktarılmaktadır. Sonraki okuma kırk ikinci bayttan başlamakta, uzunluk otuz altı olarak okunmakta ve ikinci kayıt aynı yolla kurulmaktadır. Gösterici dosya sonuna geldiğinde okuma tamamlanmaktadır.

2

Record (kayıt) uzunluğu ve başlıklar

Kayıt boyunun sektör düzenine uydurulması

KARAR

Record (kayıt) uzunluğunun seçilmesi

Karar field (alan) boylarının toplamıyla başlar, orada bitmez

Sektör boyu ikinin kuvveti olan bir diskte hizalama önem taşır

Kayıt boyu	512 baytlık sektöre sığan	Sektör sınırını aşan kayıt
30 bayt	17 kayıt + 2 bayt artık	var
32 bayt	16 kayıt, tam	yok

İki baytlık dolgu, ikinci bir erişimin önlenmesi karşılığındadır

Alan uzunlukları değişkense karar zorlaşır

- En büyük değerlere göre ayrılan yer güvenlidir ama savurgandır

Record (kayıt) uzunluğunun seçimi, field (alan) boylarının toplamıyla başlar ancak orada bitmez. Alanları toplamı otuz bayt olan bir kayıt, sektör boyu ikinin kuvveti olan bir diskte otuz iki bayta tamamlandığında bir sektöre tam sayıda kayıt yerleşir ve hiçbir kayıt iki sektöre bölünmez. İki baytlık dolgu, ikinci bir erişimin önlenmesi karşılığında ödenmiş olur. Alan uzunluklarının değişken olduğu durumlarda karar zorlaşır. En büyük değerlere göre ayrılan yer güvenlidir ama savurgandır.

3

Record (kayıt) ötesi

Self-describing file (kendini tanımlayan dosyalar), metadata ve etiketler

KAVRAM

Abstract data model (soyut veri modeli)

Veriyi diskteki dizilişinden bağımsız, uygulamanın gördüğü biçimiyle düşünmek

İlk dönemde veri teypteki görünüşüyle düşünülmüştür

- Field (alan) ve record (kayıt) dizisi

Fotoğraf, ses ve belge bu kalıba sığmaz

- Bir fotoğraf kayıt dizisi değil, piksel ızgarasıdır

Önce uygulamanın veriden ne beklediği tanımlanır

Diskteki fiziksel biçim bu tanıma göre sonradan seçilir

Abstract data model (soyut veri modeli), verinin diskte nasıl dizildiğinden bağımsız olarak, uygulamanın onu nasıl gördüğüne göre tanımlanmasıdır. Dosya işlemenin ilk dönemlerinde veri teypte ya da delikli kartta göründüğü gibi, field (alan) ve record (kayıt) dizisi olarak düşünülmüştür. Fotoğraf, ses ve belge gibi verilerin işlenmeye başlamasıyla bu bakış yetersiz kalmıştır: bir fotoğraf, art arda dizilmiş kayıtlar değil, satır ve sütunlardan oluşan bir piksel ızgarasıdır. Soyut veri modelinde önce uygulamanın veriden ne beklediği tanımlanır; verinin diskteki fiziksel biçimi bu tanıma hizmet edecek biçimde sonradan seçilir.

YAPI

Self-describing file (kendini tanımlayan dosya)

Başlığa field (alan) adları, alan genişlikleri ve alan sayısı konur

Okuyan program yapıyı dosyanın kendisinden öğrenir

- Örnek: CSV dosyasının ilk satırı sütun adlarını taşır
- Örnek: Parquet dosyası şemasını kendi içinde taşır

Kazanç: aynı program farklı yapıdaki dosyaları okuyabilir

Bedel: program, dosyadaki tanımı yorumlamak zorundadır

Dosyanın başlığına field (alan) adları, alan genişlikleri ve record (kayıt) başına alan sayısı konduğunda dosya kendini tanımlar duruma gelir. Böyle bir dosyayı okuyan program, alan yapısını kendi içinde taşımak zorunda kalmaz, yapıyı dosyadan öğrenir. Günlük bir örnek CSV dosyasıdır: ilk satır sütun adlarını taşıdığından bir tablo programı dosyayı hiçbir ön bilgi olmadan sütunlarına ayırabilir. Veri analitiğinde kullanılan Parquet biçimi ise sütunların adlarını ve tiplerini, yani şemasını, dosyanın içinde saklar. Kazanç, aynı programın farklı yapıdaki dosyalarla çalışabilmesidir. Bedeli, programın dosyadaki tanımı okuyup yorumlayacak biçimde yazılması gerekliliğidir.

KAVRAM

Metadata

Metadata: dosyadaki asıl veriyi betimleyen veri

Örnek: telefonla çekilen bir fotoğraf

- Çekim tarihi, telefon modeli, GPS konumu, pozlama süresi
- JPEG dosyasının içinde, EXIF bölümünde saklanır

Dosya kopyalanınca metadata da onunla birlikte gider

- Bu yüzden ayrı bir dosyada değil, dosyanın kendi içinde tutulur

Ortak biçimler: EXIF (fotoğraf), ID3 (MP3), PDF belge bilgisi

Metadata, dosyadaki asıl veriyi betimleyen veridir. Telefonla çekilen bir fotoğrafta asıl veri piksellerdir; çekim tarihi, telefonun modeli, fotoğrafın çekildiği yerin GPS konumu ve pozlama süresi ise metadata'dır. JPEG dosyalarında bu bilgiler dosyanın başındaki EXIF bölümünde saklanır. Metadata'nın ayrı bir dosyada değil, verinin kendi dosyasında tutulmasının nedeni, dosya kopyalandığında ya da paylaşıldığında betimlemenin de onunla birlikte gitmesidir. Bunun gündelik bir sonucu vardır: paylaşılan bir fotoğraf, çekildiği yerin konumunu da taşıyabilir; sosyal medya platformları bu nedenle yüklenen fotoğraflardaki konum bilgisini siler. Belirli bir veri türüyle çalışan topluluklar metadata için ortak biçimler üzerinde anlaşır: fotoğrafta EXIF, MP3 dosyalarında şarkı adı ve sanatçıyı taşıyan ID3, PDF belgelerinde yazar ve başlık bilgisi bunun örnekleridir.

ÖRNEK

PNG dosyasının ilk baytları

```
89 50 4E 47 0D 0A 1A 0A                imza: dosya bir PNG
00 00 00 0D  49 48 44 52               uzunluk 13 · etiket IHDR
00 00 03 20  00 00 02 58  08 02 ...    800 × 600 piksel, 8 bit, RGB
15 14 15 27                            CRC
00 00 00 12  74 45 58 74               uzunluk 18 · etiket tEXt
Author 00 Ayse Yilmaz                  anahtar = değer
3D 83 9C 91                            CRC
00 01 2F 4C  49 44 41 54               uzunluk 77 644 · etiket IDAT
78 9C ...                              sıkıştırılmış pikseller
00 00 00 00  49 45 4E 44  AE 42 60 82  uzunluk 0 · etiket IEND
```

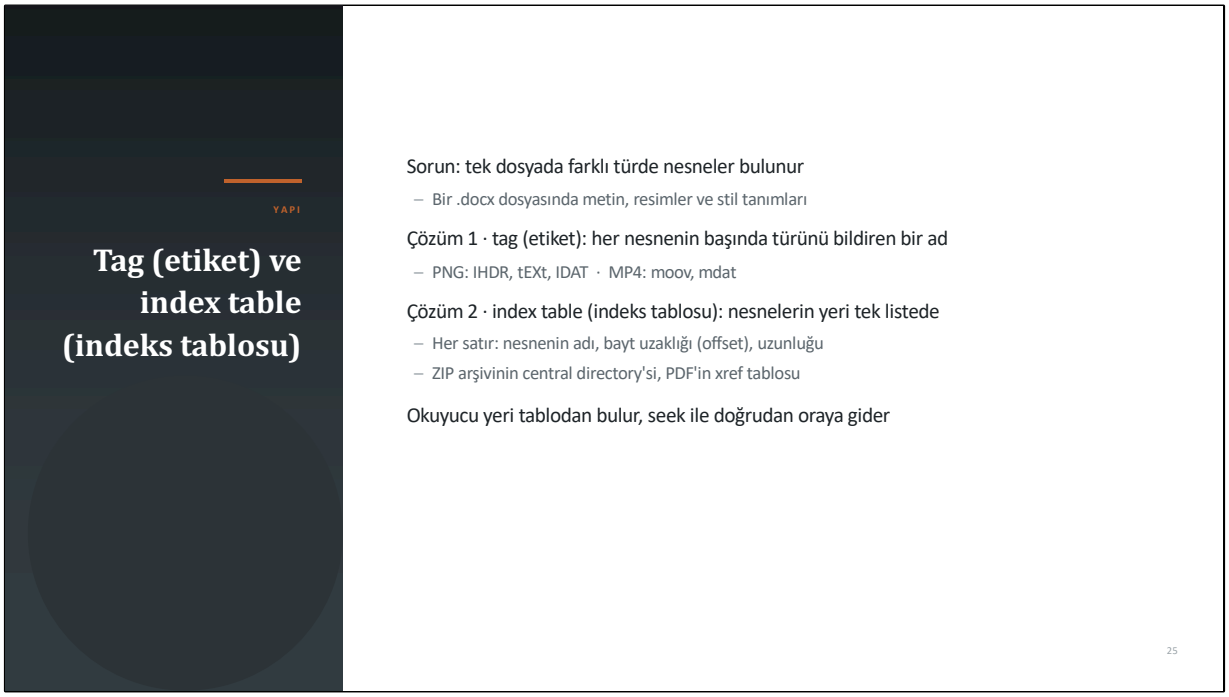
Her chunk (parça): uzunluk + tag (etiket) + veri + CRC

Tanımadığı etiketi gören program uzunluk kadar atlar

Sayılar big-endian: byte order (bayt sırası) standartta sabit

24

Bir PNG dosyası sekiz baytlık bir imzayla başlar; bu imza, dosyanın gerçekten PNG olduğunu ve aktarım sırasında satır sonu baytlarının bozulmadığını denetlemeye yarar. Ardından dosya chunk (parça) adı verilen bölümlerden oluşur. Her parça dört baytlık bir uzunlukla başlar, dört harflik bir tag (etiket) ile türünü bildirir, veriyi taşır ve dört baytlık bir CRC ile biter. IHDR parçası görüntünün boyutlarını, bit derinliğini ve renk türünü taşır; bu örnekte görüntü 800 × 600 piksel, 8 bitlik RGB'dir. tEXt parçası anahtar = değer biçiminde metin metadata'sı taşır, burada yazar adıdır. IDAT parçaları sıkıştırılmış pikselleri, IEND ise dosyanın sonunu bildirir. Bu yapı, birinci bölümdeki length indicator (uzunluk göstergesi) fikrinin bir dosya biçimine uygulanmış hâlidir: okuyan program tanımadığı bir etiketle karşılaştığında uzunluk kadar bayt atlayarak bir sonraki parçaya geçebilir. Bu sayede biçime yeni parça türleri eklendiğinde eski programlar dosyayı okumayı sürdürür. Tüm sayılar big-endian yazıldığından dosya, hangi işlemcide üretilirse üretilsin aynı biçimde okunur.



Gündelik dosyaların çoğu tek tür veri taşımaz. Bir .docx belgesinde metin, gömülü resimler ve stil tanımları; bir MP4 dosyasında görüntü, ses ve altyazı bir arada bulunur. Bu durumda her nesnenin ne olduğunun ve dosyanın neresinde durduğunun dosyada belirtilmesi gerekir. Birinci yol, her nesnenin başına türünü bildiren kısa bir ad, yani tag (etiket) konmasıdır. PNG'deki IHDR ve IDAT, MP4'teki moov ve mdat bu türden etiketlerdir. İkinci yol, nesnelerin adlarının, bayt uzaklıklarının ve uzunluklarının tek bir index table (indeks tablosunda) toplanmasıdır. Bir .docx dosyası aslında bir ZIP arşividir ve ZIP biçimi, arşivin sonunda her dosyanın yerini gösteren bir merkezî dizin taşır. Word, belgeyi açarken bu tablodan metnin bulunduğu document.xml'in yerini okur ve seek ile doğrudan oraya gider; arşivin tamamını baştan taramaz. PDF'teki xref tablosu da aynı görevi görür.

YAKLAŞIM

Gösterimden bağımsız erişim

Uygulama, verinin diskteki biçimini bilmek zorunda değildir

Örnek: bir fotoğraf düzenleyici PNG, JPEG ya da HEIC açar

- Üçü de bellekte aynı piksel dizisine çevrilir
- Düzenleme kodu yalnızca bu piksel dizisiyle çalışır

Biçimi çözen kod ayrı bir katmandadır: libpng, libjpeg gibi kütüphaneler

Kazanç 1: dosya biçimi değişse de uygulama kodu değişmez

Kazanç 2: yeni biçim için yalnızca okuyan ve yazan modül eklenir

- Tarayıcılara WebP ve AVIF desteği bu yolla eklendi

Gösterimden bağımsız erişimde uygulama, veriyi kendi beklediği bellek biçiminde alır; dosyadaki biçimin ne olduğu ayrı bir katmanın işidir. Bir fotoğraf düzenleyici PNG, JPEG ya da HEIC dosyası açabilir. Bu biçimlerin diskteki düzenleri birbirinden çok farklıdır, ancak üçü de bellekte aynı piksel dizisine çevrilir ve kırpma, parlaklık ayarı gibi düzenleme işlemleri yalnızca bu piksel dizisi üzerinde yazılır. Biçimi çözen ve yeniden yazan kod libpng ya da libjpeg gibi ayrı kütüphanelerde durur. Bu ayrımın iki kazancı vardır. Birincisi, dosya biçimi değiştiğinde ya da yeni bir sürümü çıktığında uygulamanın kendi kodu değişmez. İkincisi, yeni bir biçimin desteklenmesi yalnızca o biçimi okuyan ve yazan bir modülün eklenmesini gerektirir. Web tarayıcılarına WebP ve AVIF desteği bu yolla, sayfaları gösteren kod değiştirilmeden eklenmiştir.

4

Portability (taşınabilirlik) ve standartlaşma

Dosyanın başka bir ortamda okunabilmesi

ENGEL

İşletim sistemi ve dil farkları

İşletim sistemi farkı: satır sonu

- Windows satır sonuna iki bayt yazar: \r\n (CR LF)
- Linux ve macOS tek bayt yazar: \n (LF)
- Aynı metin dosyası iki sistemde farklı boydadır

Dil farkı: verinin dosyaya yazılış biçimi

- Java DataOutputStream sayıları her zaman big-endian yazar
- C++ fwrite yapıyı bellekteki gibi yazar: yerel bayt sırası ve dolgu baytlarıyla
- Python pickle dosyasını yalnızca Python okuyabilir

Taşınabilirliği bozan etkenlerin ilki işletim sistemi farklarıdır. En bilinen örnek satır sonudur: Windows metin kipinde her satırın sonuna satır başı ve satır atlama olmak üzere iki bayt yazar, Linux ve macOS ise yalnızca satır atlama baytını yazar. Aynı metin dosyası bu yüzden iki sistemde farklı boyda olur ve bir sistemde hesaplanan bayt konumları ötekinde tutmaz. İkinci etken programlama dilidir, çünkü dil verinin dosyaya yazılış biçimini belirler. Java'nın DataOutputStream sınıfı sayıları her zaman big-endian sırayla yazar. C++'ta fwrite ile yazılan bir yapı ise bellekteki görüntüsüyle, yani makinenin yerel bayt sırası ve derleyicinin eklediği dolgu baytlarıyla birlikte dosyaya geçer. Bu nedenle Java ile yazılmış bir ikili dosya C++ programında, ek dönüşüm yapılmadan, yanlış sayılarla okunur. Python'un pickle biçimi ise yalnızca Python tarafından okunabilir.

ENGEL

Makine mimarisi farkları

- Byte order (bayt sırası) makineden makineye değişir
  - Aynı program aynı sayıyı farklı sırayla yazar
- Yapı üyelerinin bellekteki yerleşimi de değişir
  - Aynı yapı 16, 8 ya da 6 bayt tutabilir
- Metin kodlaması ayrışır: UTF-8, UTF-16, Windows-1254
  - UTF-8 ile yazılan ş (C5 9F), Windows-1254 ile okunursa ÅŸ görünür
- Türkçe karakterler bu sorunun en görünür örneğidir

Üçüncü ve en ince etken makine mimarisi farklarıdır. Çok baytlı sayıların byte order (bayt sırası) makineden makineye değişir; aynı program aynı sayıyı farklı sırayla yazar. Yapı üyelerinin bellekteki yerleşimi de değişebilir: aynı yapı bir makinede on altı, başkasında sekiz, bir üçüncüsünde altı bayt yer tutabilir. Metin kodlaması da ayrışır. Bugün web'in fiilî standardı UTF-8'dir; Windows'un iç yapısında UTF-16, eski Türkçe Windows uygulamalarında ise Windows-1254 kod sayfası kullanılır. UTF-8 ile yazılmış ş harfi iki bayttır, C5 ve 9F. Dosyayı Windows-1254 varsayarak okuyan bir program bu iki baytı ayrı ayrı iki karakter sanar ve ekranda ÅŸ gösterir. Türkçe karakterlerin e-postalarda, CSV dosyalarında ve altyazılarda bozuk görünmesinin nedeni budur.

ÇÖZÜM 1

Fiziksel standart

Bir record (kayıt) biçimi üzerinde anlaşmak ve ona bağlı kalmak

Fiziksel standart, ortamdan bağımsız olarak aynı temsil edilir

PNG örneği: imza, chunk (parça) yapısı, big-endian sayılar

Standardın kalıcı olması için iki koşul

- Yeterince basit olmalı
- Genişletilebilir tasarlanmalı

Taşınabilirliğin sağlanmasında ilk yol, fiziksel bir record (kayıt) biçimi üzerinde anlaşmak ve ona bağlı kalmaktır. Fiziksel standart, hangi dil, makine ya da işletim sistemi kullanılırsa kullanılsın aynı biçimde temsil edilen standarttır. PNG, dosyanın imzasını, parçaların yapısını ve sayıların big-endian yazılacağını belirlediği için bunun iyi bir örneğidir: hangi makinede üretilirse üretilsin her PNG dosyası her yerde aynı biçimde okunur. Standardın kalıcı olması için yeterince basit olması ve genişletilebilir tasarlanması gerekir.

5

Direct access ve hashing girişi

Record (kaydın) yerinin anahtardan hesaplanması

NEDEN

Aramanın bedeli

Sıralı dosyada arama ortalama n / 2 erişim

Binary search (ikili arama) $\log_2 n$ erişim, ama dosya sıralı kalmalı

- Sıralılığı korumak her eklemede record (kaydırma) demektir

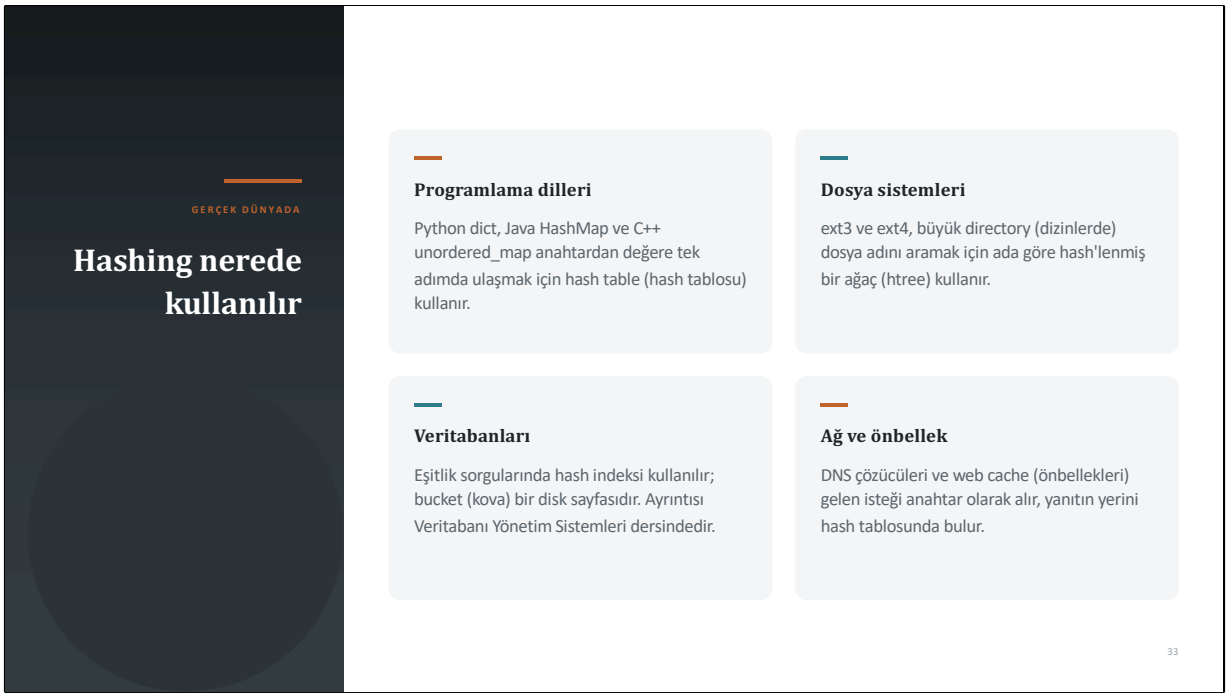
Bir milyon kayıt: 500 000 ya da 20 erişim

- Diskte 20 erişim bile yaklaşık 20 × 10 ms

Hedef: anahtar verildiğinde tek erişim

Bedel: sıralı okuma ve range query (aralık sorgusu) yitirilir

Direct access (doğrudan erişim) yöntemlerinin çıkış noktası aramanın maliyetidir. Sıralı bir dosyada bir record (kayıt) ortalama olarak dosyanın yarısı okunduktan sonra bulunur. Binary search (ikili arama) erişim sayısını logaritmik düzeye indirir, ancak dosyanın her an sıralı tutulmasını gerektirir ve sıralılığın korunması her eklemede kayıtların kaydırılması anlamına gelir. Bir milyon kayıtlık bir dosyada ikili arama yaklaşık yirmi erişim gerektirir; disk üzerinde bu, saniyenin beşte birine yaklaşan bir süredir. Hashing'in hedefi erişim sayısını bire indirmektir. Bu kazancın karşılığında kayıtların anahtar sırası yitirilir, dolayısıyla sıralı okuma ve range query (aralık sorgusu) bu yapıyla yapılamaz.



Hashing, anahtardan değere tek adımda ulaşılması gereken her yerde kullanılmaktadır. Programlama dillerinin sözlük ve eşleme yapıları bellekteki hash table (hash tablolarıdır). File system (dosya sistemlerinde) çok sayıda girdi içeren directory (dizinlerde) dosya adının doğrusal taranması pahalı olduğundan ext3 ve ext4 dosya adını hash'leyerek aranan girdinin bulunduğu bloğa doğrudan iner. Veritabanı yönetim sistemlerinde eşitlik koşullu sorgular için hash indeksleri kullanılır ve bucket (kova) bir disk sayfasına karşılık gelir. Cache (önbelleklerde) istek anahtar olarak alınır ve yanıtın yeri hash tablosunda aranır.

ALGORITMA

Direct access (doğrudan erişimli) dosyalar

Record (kayda) tek adımda ulaşılmasını sağlayan dosyalar

Erişim için kaydın saklandığı yerin adresi belirlenmelidir

Adresin anahtardan belirlenmesi: doğrudan adresleme

İki yol vardır

- Anahtar değerin doğrudan adres olması
- Anahtar değerin bir adrese dönüştürülmesi

Direct access (doğrudan erişimli) dosyalar, bir record (kayda) tek adımda ulaşılmasını sağlayan dosyalardır. Böyle bir dosyada kayda erişebilmek için kaydın saklandığı yerin adresinin belirlenmesi gerekir. Adresin kaydın anahtarından belirlenmesine doğrudan adresleme denir. Kayda iki yolla doğrudan erişilebilir: anahtar değerin doğrudan bir adres olması ya da anahtar değerin bir adrese dönüştürülmesi. Birinci yol yalnızca anahtar uzayının küçük ve yoğun olduğu durumlarda uygulanabilir. İkinci yol ise hash function (hash fonksiyonları) aracılığıyla genel duruma taşınır ve dönemin ikinci yarısında incelenecek statik ve dynamic hashing (dinamik hashing) yöntemlerinin temelini oluşturur.

ALGORITMA · 1

Anahtarın doğrudan adres olması

Arama tek adımda sonuçlanır

Örnek: kodu 1 ile 100 arası olan ürünler

- Kodu 35 olan ürün 35. konumdadır

Sakınca: adres uzayı anahtar uzayı kadar büyük olmalıdır

- Kodlar 1000 ile 9999 arası olsaydı, 100 ürün için 9000 konum
- Kullanılmayan alan kullanılandan çok daha geniş olurdu

Anahtar değerin doğrudan adres olması durumunda arama tek adımda sonuçlanır. Kodları bir ile yüz arasında olan ürünlerin saklandığı bir dosyada, kodu otuz beş olan ürün dosyanın otuz beşinci konumunda bulunur. Yöntemin sakıncası, adres uzayının anahtar uzayı kadar büyük olmak zorunda kalmasıdır. Aynı yüz ürünün kodları bin ile dokuz bin dokuz yüz doksan dokuz arasında olsaydı, dosyada kullanılmayan çok geniş bir alan ayrılması gerekirdi.

ALGORITMA · 2

Anahtarın adrese dönüştürülmesi

Standart bir yol yoktur, yöntem veriye göre değişir

Havayolu rezervasyon örneği

- Her gün 1 ile 999 arası uçuş, günler 1 ile 366 arası numaralı
- Uçuş no || gün no → 001001 ile 999366 arası
- Gün no || uçuş no → 001001 ile 366999 arası

Adres uzayı anahtar uzayını karşılayamazsa hashing'e geçilir

Anahtar değerin tek bir adrese dönüştürülmesinin standart bir yolu yoktur. Yöntem veriye göre değişir. Havayolu rezervasyon örneğinde her gün bir ile dokuz yüz doksan dokuz arasında uçuş bulunmakta, günler bir ile üç yüz altmış altı arasında numaralanmaktadır. Uçuş numarası ile gün numarası ardışık yazıldığında adres elde edilir. Sıralamanın değiştirilmesi adres aralığının sınırlarını da değiştirir. Bu tür düzenlemelerle adres uzayı anahtar uzayını karşılayamadığında hash algoritmalarına başvurulur.

BAĞLANTI

Hash adresi bir RRN'dir

Bölüm 1: sabit record (kaydın) yeri = RRN × kayıt boyu

Hash function (hash fonksiyonu) anahtardan bu RRN'yi üretir

- Bayt konumu yine çarpmayla bulunur

Direct access (doğrudan erişimli) dosya fixed-length record (sabit uzunluklu kayıt) ister

- Değişken kayıtta adres → konum tablosu gerekir

Bölüm 2: bir sektöre 16 record (kayıt) sığar → bir adreste birden çok kayıt

- Bu fikir algoritma hattında bucket olarak döner

Hashing'in dosya düzeyindeki karşılığı haftanın ana hattıyla doğrudan ilişkilidir. Fixed-length record (sabit uzunluklu kayıtlarda) bir record (kaydın) bayt konumu, relative record number (göreli kayıt numarasının) kayıt boyuyla çarpılmasıyla bulunmaktaydı. Hash function (hash fonksiyonunun) ürettiği adres de bir göreli kayıt numarasıdır; bayt konumu yine aynı çarpmayla hesaplanır. Bu nedenle direct access (doğrudan erişimli) dosyalar sabit uzunluklu kayıtlarla kurulur. Variable-length record (değişken uzunluklu kayıtlarda) adresi konuma çeviren ek bir tablo gerekir. İkinci bölümde bir sektöre birden çok kaydın yerleştirilmesi ise bir adreste birden çok kaydın tutulması fikrinin kaynağıdır ve algoritma hattında bucket kavramı olarak yeniden karşılaşılacaktır.

KAVRAM

Hashing algoritmasının iki bileşeni



iki anahtarın aynı adrese düşmesi kaçınılmazdır: çarpışma

Hash function (hash fonksiyonu): anahtardan adrese geçişi sağlar

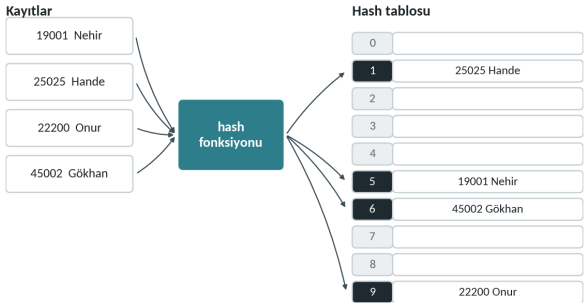
Collision resolution (çarpışma çözümleme): aynı adresi üreten anahtarları yerleştirir

38

Hashing algoritmasının iki temel bileşeni vardır. Birincisi, record (kaydın) anahtar bilgisinden adres bilgisine geçişi sağlayan hash function (hash fonksiyonudur). İkincisi, farklı anahtar değerlerin aynı adresi üretmesi durumunda kullanılan collision resolution (çarpışma çözümleme) yöntemidir. Anahtar uzayı adres uzayından geniş olduğu için iki anahtarın aynı adrese düşmesi kaçınılmazdır. Bu nedenle hash fonksiyonu tek başına yeterli değildir.

KAVRAM

Hash function (hash fonksiyonu) ve hash table (hash tablosu)



Dönen adres olası adrestir, tek adres güvencesi yoktur

39

Hash function (hash fonksiyonu), geniş aralıktaki anahtarları daha küçük bir adres aralığına dönüştürür. Fonksiyondan dönen adres olası adrestir. Her anahtar için tek adres güvencesi bulunmaz, çünkü adres uzayı ile anahtar uzayı aynı değildir. Şekilde dört record (kaydın) hash fonksiyonundan geçirilerek hash table (hash tablosundaki) yerlerine yerleştirilmesi izlenmektedir. Adresleri çakışan kayıtlar, çeşitli yöntemlerle elde edilen başka bir adrese yazılır.

ÖLÇÜT

Hash function (hash fonksiyonunun) nitelikleri

- Anahtarları adres aralığında düzgün dağıtmalıdır
- Hızlı çalışmalıdır
- Az collision (çarpışmaya) yol açmalıdır
- Sayısal anahtar değeri gerektirir
  - Anahtar karakterse sayısal karşılıkları kullanılır
- Record (kayıtlar), fonksiyondan dönen adreslerde saklanır

Bir hash function (hash fonksiyonunda) bulunması gereken nitelikler sınırlıdır ve birbiriyle ilişkilidir. Fonksiyon anahtarları adres aralığına düzgün dağıtmalı, hızlı çalışmalı ve az collision (çarpışmaya) yol açmalıdır. Hash fonksiyonu sayısal anahtar değeri gerektirir. Anahtarın karakter olması durumunda karakterlerin sayısal karşılıkları kullanılır. Direct access (doğrudan erişimli) dosyalarda record (kayıtlar), anahtarın fonksiyondan geçirilmesiyle elde edilen adreslerde saklanır. Bu nitelikler birbirini dengelemektedir. Daha düzgün dağılım üreten bir fonksiyon çoğu zaman daha fazla işlem gerektirir. Dosya bağlamında ise fonksiyonun hesap süresi bir disk erişiminin yanında önemsiz kaldığından, dağılımın düzgünlüğü hızdan daha belirleyicidir.

FONKSIYON

Division method (bölme yöntemi)

$$h(x) = x \bmod n$$

n: tablo boyu Sonuç 0 ile n−1 arasındadır

En yaygın kullanılan hash function (hash fonksiyonudur)

Bölen olarak asal sayı kullanılır

- f(anahtar) = anahtar mod p, p ≤ n olan en büyük asal
- Asal bölen, anahtardaki düzenli örüntüleri adrese taşımaz

Tek bölme işlemi gerektirdiği için hız ölçütünü karşılar

En yaygın kullanılan hash function (hash fonksiyonu) division method (bölme yöntemidir). Boyutu n olan bir tabloda x anahtarı, sıfır ile n−1 arasında bir değerle eşleştirilir. Yöntemin bir değişkesinde bölen olarak tablo boyundan küçük ya da ona eşit en büyük asal sayı kullanılır. Asal bölen, anahtarlardaki düzenli örüntülerin adreslere taşınmasını azalttığı için yeğlenir. İki fonksiyon da tek bir bölme işlemi gerektirdiğinden hız ölçütünü karşılar.

ADIM 1 / 9

Division method (bölme yöntemi): 71

1. anahtar: 71

71 mod 11 = 5



çarpışan

71 = 11 × 6 + 5

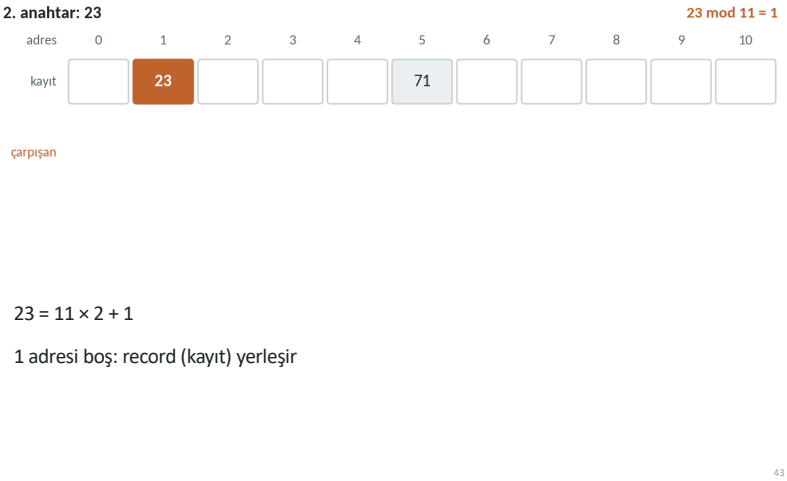
5 adresi boş: record (kayıt) yerleşir

42

Birinci adımda 71 anahtarı on bire bölünmektedir. Bölüm 6, kalan 5 olduğundan anahtarın adresi 5 olarak hesaplanmaktadır. Adres boş olduğundan record (kayıt) doğrudan kendi adresine yerleşmekte ve tek erişimle bulunabilir duruma gelmektedir. Bu adımdan sonra kullanılan adres sayısı 1, toplam collision (çarpışma) sayısı 0 olmaktadır.

ADIM 2 / 9

Division method (bölme yöntemi): 23



İkinci adımda 23 anahtarı on bire bölünmektedir. Bölüm 2, kalan 1 olduğundan anahtarın adresi 1 olarak hesaplanmaktadır. Adres boş olduğundan record (kayıt) doğrudan kendi adresine yerleşmekte ve tek erişimle bulunabilir duruma gelmektedir. Bu adımdan sonra kullanılan adres sayısı 2, toplam collision (çarpışma) sayısı 0 olmaktadır.

ADIM 3 / 9

Division method (bölme yöntemi): 73

3. anahtar: 73

73 mod 11 = 7



çarpışan

73 = 11 × 6 + 7

7 adresi boş: record (kayıt) yerleşir

44

Üçüncü adımda 73 anahtarı on bire bölünmektedir. Bölüm 6, kalan 7 olduğundan anahtarın adresi 7 olarak hesaplanmaktadır. Adres boş olduğundan record (kayıt) doğrudan kendi adresine yerleşmekte ve tek erişimle bulunabilir duruma gelmektedir. Bu adımdan sonra kullanılan adres sayısı 3, toplam collision (çarpışma) sayısı 0 olmaktadır.

ADIM 4 / 9

Division method (bölme yöntemi): 99

4. anahtar: 99

99 mod 11 = 0



çarpışan

99 = 11 × 9 + 0

0 adresi boş: record (kayıt) yerleşir

45

Dörtüncü adımda 99 anahtarı on bire bölünmektedir. Bölüm 9, kalan 0 olduğundan anahtarın adresi 0 olarak hesaplanmaktadır. Adres boş olduğundan record (kayıt) doğrudan kendi adresine yerleşmekte ve tek erişimle bulunabilir duruma gelmektedir. Bu adımdan sonra kullanılan adres sayısı 4, toplam collision (çarpışma) sayısı 0 olmaktadır.

ADIM 5 / 9

Division method (bölme yöntemi): 44



44 = 11 × 4 + 0

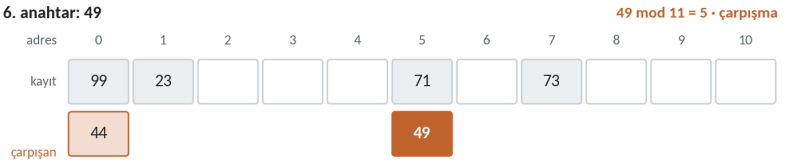
0 adresi dolu: çarpışma (1. çarpışma)

46

Beşinci adımda 44 anahtarı on bire bölünmektedir. Bölüm 4, kalan 0 olduğundan anahtarın adresi 0 olarak hesaplanmaktadır. Adres daha önce 99 anahtarıyla doldurulmuş olduğundan yeni record (kayıt) kendi adresine yerleşememektedir. Bu, dizinin bir numaralı collision (çarpışmasıdır). Çarpışan kayıt şekilde adresin altında gösterilmekte, ancak gerçek bir dosyada bu kaydın nereye yazılacağı collision resolution (çarpışma çözümleme) yöntemi tarafından belirlenmektedir. Bu adımdan sonra kullanılan adres sayısı 4, toplam çarpışma sayısı 1 olmaktadır.

ADIM 6 / 9

Division method (bölme yöntemi): 49



49 = 11 × 4 + 5

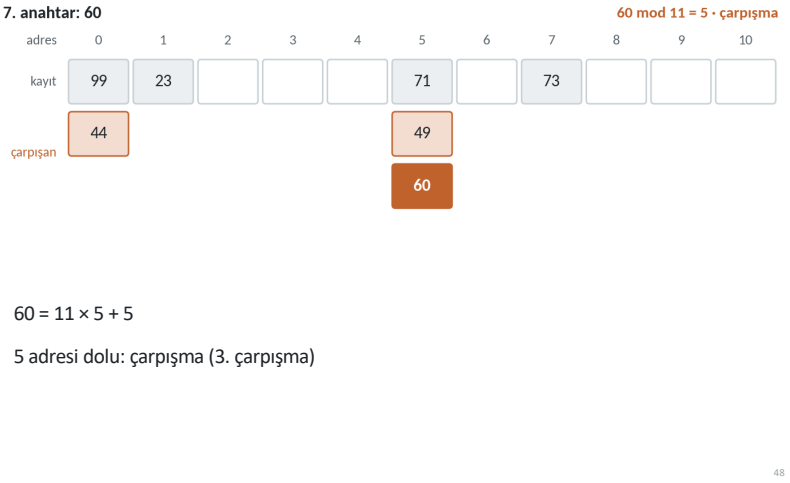
5 adresi dolu: çarpışma (2. çarpışma)

47

Altıncı adımda 49 anahtarı on bire bölünmektedir. Bölüm 4, kalan 5 olduğundan anahtarın adresi 5 olarak hesaplanmaktadır. Adres daha önce 71 anahtarıyla doldurulmuş olduğundan yeni record (kayıt) kendi adresine yerleşememektedir. Bu, dizinin iki numaralı collision (çarpışmasıdır). Çarpışan kayıt şekilde adresin altında gösterilmekte, ancak gerçek bir dosyada bu kaydın nereye yazılacağı collision resolution (çarpışma çözümleme) yöntemi tarafından belirlenmektedir. Bu adımdan sonra kullanılan adres sayısı 4, toplam çarpışma sayısı 2 olmaktadır.

ADIM 7 / 9

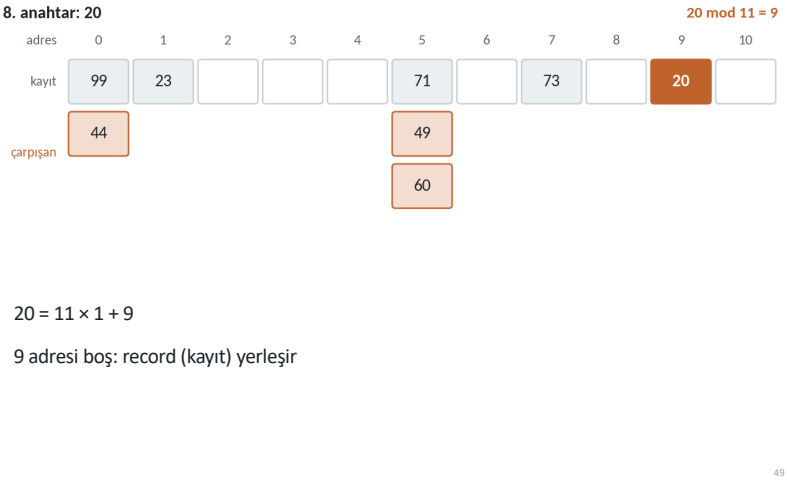
Division method (bölme yöntemi): 60



Yedinci adımda 60 anahtarı on bire bölünmektedir. Bölüm 5, kalan 5 olduğundan anahtarın adresi 5 olarak hesaplanmaktadır. Adres daha önce 71 anahtarıyla doldurulmuş olduğundan yeni record (kayıt) kendi adresine yerleşememektedir. Bu, dizinin üç numaralı collision (çarpışmasıdır). Çarpışan kayıt şekilde adresin altında gösterilmekte, ancak gerçek bir dosyada bu kaydın nereye yazılacağı collision resolution (çarpışma çözümleme) yöntemi tarafından belirlenmektedir. Bu adımdan sonra kullanılan adres sayısı 4, toplam çarpışma sayısı 3 olmaktadır.

ADIM 8 / 9

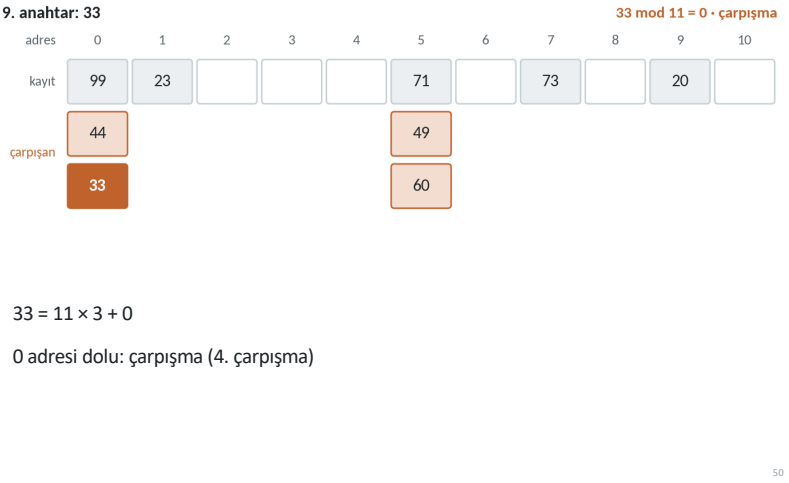
Division method (bölme yöntemi): 20



Sekizinci adımda 20 anahtarı on bire bölünmektedir. Bölüm 1, kalan 9 olduğundan anahtarın adresi 9 olarak hesaplanmaktadır. Adres boş olduğundan record (kayıt) doğrudan kendi adresine yerleşmekte ve tek erişimle bulunabilir duruma gelmektedir. Bu adımdan sonra kullanılan adres sayısı 5, toplam collision (çarpışma) sayısı 3 olmaktadır.

ADIM 9 / 9

Division method (bölme yöntemi): 33



Dokuzuncu adımda 33 anahtarı on bire bölünmektedir. Bölüm 3, kalan 0 olduğundan anahtarın adresi 0 olarak hesaplanmaktadır. Adres daha önce 99 anahtarıyla doldurulmuş olduğundan yeni record (kayıt) kendi adresine yerleşememektedir. Bu, dizinin dört numaralı collision (çarpışmasıdır). Çarpışan kayıt şekilde adresin altında gösterilmekte, ancak gerçek bir dosyada bu kaydın nereye yazılacağı collision resolution (çarpışma çözümleme) yöntemi tarafından belirlenmektedir. Bu adımdan sonra kullanılan adres sayısı 5, toplam çarpışma sayısı 4 olmaktadır.

FONKSIYON

Truncation (kesme) ve folding (katlama)

Yöntem	Örnek	Sonuç
Kesme: 4., 7., 8. basamak	21296876	976
Kesme: anlamlı bölüm	XXYYZZNNN → XXNNN	yıl + sıra
Katlama: sınırdan	321 + 456 + 987	654
Katlama: üst üste	123 + 456 + 789	258

Kesmede anahtarın belirli basamakları adres olarak alınır

Anlamlı bölüm seçimi: aynı bölümdeki öğrencilerde bölüm kodu ayırt etmez

Katlamada parçalar eldesiz toplanır

Truncation (kesme) yönteminde anahtarın belirli basamakları alınarak adres olarak kullanılır. Sekiz basamaklı bir anahtardan üç basamak seçilerek bin record (kayıtlık) bir tabloya yerleşim sağlanabilir. Yöntemin bir kullanımı, anahtarın anlamlı bölümlerinin seçilmesidir: aynı bölümün öğrencilerinden oluşan bir dosyada fakülte ve bölüm kodu ayırt edici olmadığı için dışarıda bırakılır. Folding (katlama) yönteminde ise anahtar parçalara bölünür ve parçalar eldesiz toplanır. Parçalar doğrudan ya da ters çevrilerek toplanabilir.

FONKSIYON

Mid-square (kare alma), radix ve alfanümerik

Yöntem	Örnek	Sonuç
Kare alma → orta basamaklar	$2315^2 = 5359225$	592
Radix dönüşümü	$(1234)_{11} = (1610)_{10}$	1610
Alfanümerik anahtar	"BC" → 66 + 67	133

Orta basamaklar anahtarın tüm basamaklarından etkilenir

Radix dönüşümünün ardından genellikle kesme uygulanır

Alfanümerik anahtarda önce sayısal karşılık elde edilir

Mid-square (kare alma) yönteminde anahtarın karesi hesaplanır ve sonucun orta basamakları adres olarak alınır. Ortadaki basamaklar anahtarın tüm basamaklarından etkilendiği için dağılım daha düzgün olur. Radix dönüşümünde anahtar, on tabanından başka bir tabanda yazılmış sayılarak yeniden ona çevrilir ve ardından genellikle kesme uygulanır. Alfanümerik anahtarlarda ise karakterlerin sayısal karşılıkları elde edildikten sonra bu fonksiyonlardan herhangi biri uygulanır.

ÇÖZÜMLÜ ÖRNEK

Aynı anahtar, altı fonksiyon

Yöntem	Adım adım hesap	Adres
Bölme, p = 997	472195836 = 997 × 473616 + 684	684
Kesme, 4. 7. 8. basamak	4 7 2 [1] 9 5 [8] [3] 6	183
Katlama, sınırdan	274 + 195 + 638, basamak basamak eldesiz	997
Katlama, üst üste	472 + 195 + 836, basamak basamak eldesiz	393
Kare alma, 2718	2718^2 = 7387524, ortadaki üç basamak	875
Alfanümerik, “DO”	D = 68, O = 79, 68 + 79	147

Anahtar 472195836, adres uzayı 0 ile 999

Eldesiz: her basamak ayrı toplanır, elde atılır

53

Tabloda dokuz basamaklı 472195836 anahtarı bin adreslik bir tabloya altı ayrı fonksiyonla yerleştirilmektedir. Division method (bölme yönteminde) bin sayısından küçük en büyük asal olan 997 bölen alınmakta ve kalan 684 adres olmaktadır. Kesmede dördüncü, yedinci ve sekizinci basamaklar sırayla alınarak 183 elde edilmektedir. Sınırdan folding (katlamada) anahtar üçerli parçalara bölünmekte, dış parçalar ters çevrilmekte ve birler, onlar, yüzler basamakları elde taşınmadan ayrı ayrı toplanmaktadır. Birler basamağında 4, 5 ve 8 toplamı 17 olduğundan yalnızca 7 yazılmaktadır. Üst üste katlamada parçalar ters çevrilmeden aynı biçimde toplanmaktadır. Mid-square (kare almada) 2718 sayısının karesinin ortadaki üç basamağı, alfanümerik anahtarda ise harflerin ASCII kodlarının toplamı kullanılmaktadır.

ÖLÇÜT

Collision (çarpışmanın) azaltılması

Birinci yol: probe sayısını az tutmak

- Probe: record (kaydın) boş yer bulana dek gittiği adres sayısı
- Erişimdeki adım sayısı retrieval probe olarak adlandırılır

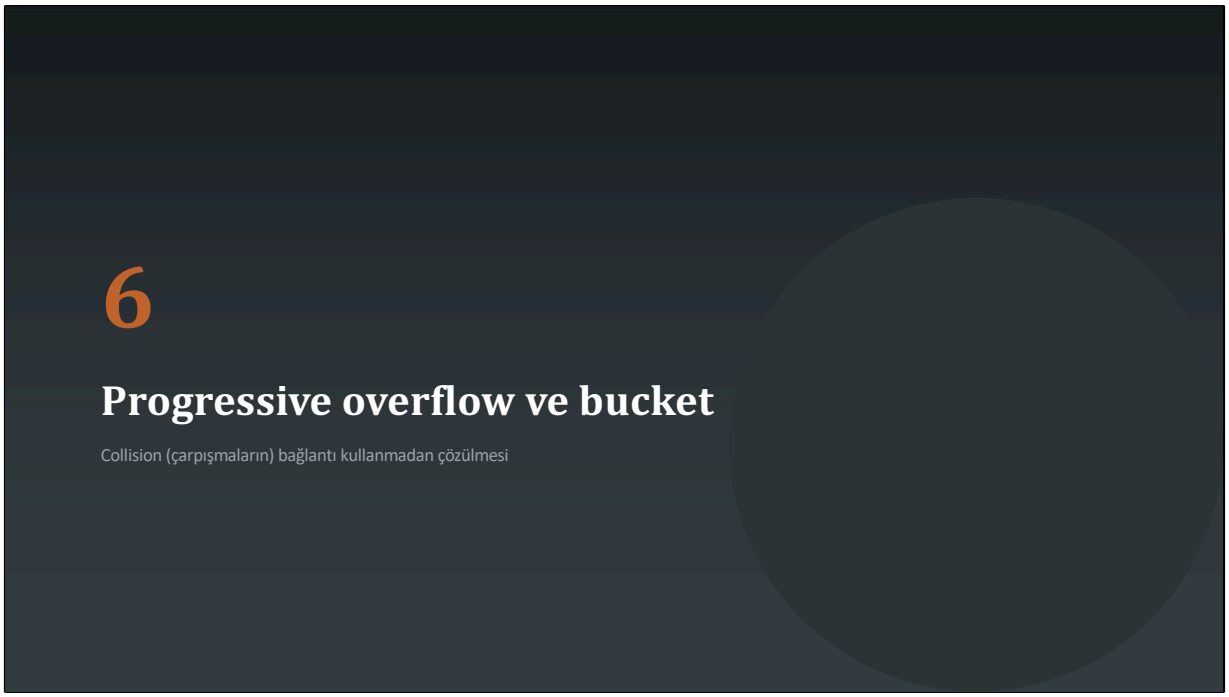
load factor (depolama oranı) = depolanan kayıt ÷ depolanabilecek kayıt

İkinci yol: oranın düşürülmesi

Oranın düşürülmesi çarpışmayı azaltır, ortadan kaldırmaz

Bu nedenle collision resolution (çarpışma çözümleme) yöntemleri gereklidir

Collision (çarpışmaların) azaltılması için iki yol vardır. Birincisi probe sayısını az tutmaktır. Probe, bir record (kaydın) yerleşeceği boş yeri bulana dek gittiği adres sayısıdır ve bu sayının küçüklüğü algoritmanın iyi çalıştığını gösterir. Çarpışmalar arttıkça bir kayda erişim için gereken adım sayısı da artar. İkinci yol load factor (depolama oranını) düşürmektir. Oran, depolanan kayıt sayısının depolanabilecek kayıt sayısına bölünmesiyle bulunur. Oranın düşürülmesi çarpışmayı azaltır ama ortadan kaldırmaz.



Bu bölümde bağlantı kullanmadan çözülmesi ele alınmaktadır. Yöntem, dolu bir adresten sonraki adreslere ilerlemeye dayanır. Bölümün ikinci yarısında, bir adrese birden çok record (kayıt) yazma fikri tanıtılmakta ve haftanın bloklama kavramıyla ilişkilendirilmektedir.

NEDEN

Collision (çarpışma) kaçınılmazdır

Doğum günü sorusu: 23 kişide ortak gün olasılığı %50'yi aşar

1000 adreslik tabloda 38 record (kayıttan) sonra aynı durum

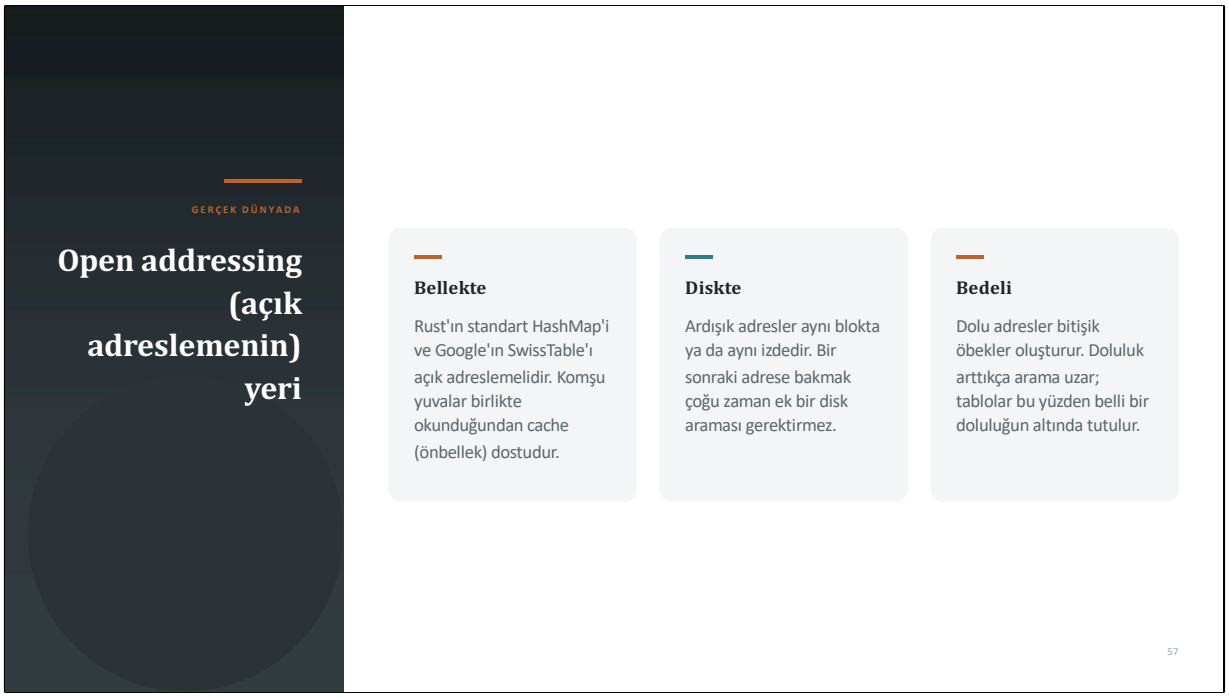
- Tablo neredeyse boşken bile çarpışma beklenir

Her hash yöntemine bir collision resolution (çarpışma çözümleme) yöntemi eşlik eder

Open addressing (açık adresleme): kayıt tablonun içinde başka bir yere konur

- Ek bellek, işaretçi ya da link field (bağlantı alanı) gerekmez

Hash function (hash fonksiyonu) ne kadar iyi seçilirse seçilsin collision (çarpışma) kaçınılmazdır. Doğum günü sorusu bunun bilinen örneğidir: yirmi üç kişilik bir toplulukta iki kişinin aynı gün doğmuş olma olasılığı yüzde elliyi geçer. Aynı hesap bin adreslik bir tablo için yapıldığında, otuz sekiz record (kayıttan) sonra en az bir çarpışma olasılığı yüzde elliyi aşar; tablo neredeyse boşken bile çarpışma beklenmelidir. Bu nedenle her hash yöntemine bir collision resolution (çarpışma çözümleme) yöntemi eşlik eder. Bu bölümdeki yöntemler open addressing (açık adresleme) ailesindendir: çarpışan kayıt tablonun içinde başka bir adrese yerleştirilir ve ek bir bellek alanı ya da link field (bağlantı alanı) kullanılmaz.



Progressive overflow (adım adım ilerleyen taşma) en yalın open addressing (açık adresleme) yöntemidir ve bugün de bellek içi hash table (hash tablolarında) ailesinin temel fikri olarak kullanılmaktadır. Rust dilinin standart eşleme yapısı ve Google'ın SwissTable tasarımı açık adreslemelidir; komşu yuvaların birlikte okunması işlemci önbelleğinden yararlanmayı sağlar. Disk üzerinde de ardışık adreslerin aynı blokta ya da aynı izde bulunması, bir sonraki adrese bakmanın ek bir arama gerektirmemesi anlamına gelir. Yöntemin bedeli kümelenmedir: doluluk arttıkça ortalama arama uzunluğu hızla büyür ve tablolar bu nedenle belirli bir doluluk oranının altında tutulur.

ALGORITMA

Progressive overflow (adım adım ilerleyen taşma)

Kullanımı en basit collision resolution (çarpışma çözümleme) yöntemlerinden biri

Home address (ev adresi) doluysa bir sonraki adrese bakılır

- Boş yer bulunana dek ilerlenir
- Record (kayıt), bulunan ilk boş yere yazılır

Tablonun sonuna gelindiğinde başa dönülür

- Tablo dairesel bir bütün gibi düşünülür

Arama da aynı yolu izler: ev adresinden başlar

Progressive overflow (adım adım ilerleyen taşma), kullanımı en basit collision resolution (çarpışma çözümleme) yöntemlerinden biridir. Eklenecek record (kaydın) home address (ev adresi) doluysa bir sonraki adrese bakılır. Orası da doluysa boş bir yer bulunana dek ilerlenir ve kayıt bulunan ilk boş yere yazılır. Tablonun sonuna gelindiğinde başa dönülür. Tablo dairesel bir bütün gibi düşünülür. Arama da aynı yolu izler: ev adresinden başlanır ve kayıt bulunana ya da boş bir adrese rastlanana dek ilerlenir.

ADIM 1 / 9

Progressive overflow: 35



35 mod 10 = 5

Home address (ev adresi) 5 boş: 1 probe

59

Birinci adımda 35 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 5 değeridir. Ev adresi boş olduğundan record (kayıt) tek probe ile yerleşmektedir. Tabloda henüz hiç kayıt bulunmadığından ev adresinin boş bulunma olasılığı yüksektir. Doluluk arttıkça bu olasılık hızla düşmektedir. Bu adımla birlikte toplam probe sayısı 1 olmaktadır.

ADIM 2 / 9

Progressive overflow: 43



43 mod 10 = 3

Home address (ev adresi) 3 boş: 1 probe

60

İkinci adımda 43 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 3 değeridir. Ev adresi boş olduğundan record (kayıt) tek probe ile yerleşmektedir. Tabloda bir kayıt bulunduğundan ev adresinin boş bulunma olasılığı yüksektir. Doluluk arttıkça bu olasılık hızla düşmektedir. Bu adımla birlikte toplam probe sayısı 2 olmaktadır.

ADIM 3 / 9

Progressive overflow: 70



70 mod 10 = 0

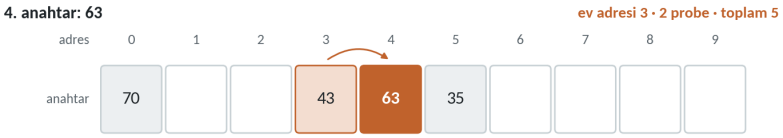
Home address (ev adresi) 0 boş: 1 probe

61

Üçüncü adımda 70 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 0 değeridir. Ev adresi boş olduğundan record (kayıt) tek probe ile yerleşmektedir. Tabloda iki kayıt bulunduğundan ev adresinin boş bulunma olasılığı yüksektir. Doluluk arttıkça bu olasılık hızla düşmektedir. Bu adımla birlikte toplam probe sayısı 3 olmaktadır.

ADIM 4 / 9

Progressive overflow: 63



63 mod 10 = 3

Dolu: 3 → 4 adresine, 2 probe

62

Dörtüncü adımda 63 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 3 değeridir. Ev adresi doludur. Yöntem bir sonraki adrese geçmekte ve boş yer bulunana dek ilerlemektedir. Sırasıyla 3 adresleri dolu bulunmakta, record (kayıt) 4 adresine yazılmaktadır. Bu yerleşim iki probe gerektirmektedir. Bu adımla birlikte toplam probe sayısı 5 olmaktadır.

ADIM 5 / 9

Progressive overflow: 30



30 mod 10 = 0

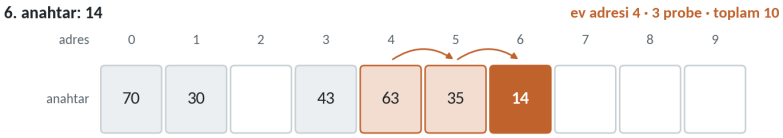
Dolu: 0 → 1 adresine, 2 probe

63

Beşinci adımda 30 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 0 değeridir. Ev adresi doludur. Yöntem bir sonraki adrese geçmekte ve boş yer bulunana dek ilerlemektedir. Sırasıyla 0 adresleri dolu bulunmakta, record (kayıt) 1 adresine yazılmaktadır. Bu yerleşim iki probe gerektirmektedir. Bu adımla birlikte toplam probe sayısı 7 olmaktadır.

ADIM 6 / 9

Progressive overflow: 14



14 mod 10 = 4

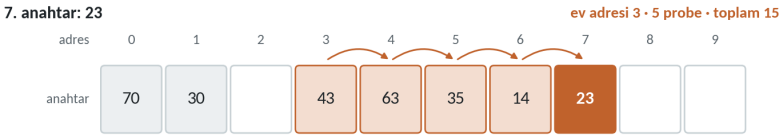
Dolu: 4, 5 → 6 adresine, 3 probe

64

Altıncı adımda 14 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 4 değeridir. Ev adresi doludur. Yöntem bir sonraki adrese geçmekte ve boş yer bulunana dek ilerlemektedir. Sırasıyla 4, 5 adresleri dolu bulunmakta, record (kayıt) 6 adresine yazılmaktadır. Bu yerleşim üç probe gerektirmektedir. Bu adımla birlikte toplam probe sayısı 10 olmaktadır.

ADIM 7 / 9

Progressive overflow: 23



23 mod 10 = 3

Dolu: 3, 4, 5, 6 → 7 adresine, 5 probe

65

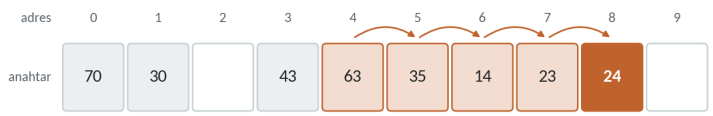
Yedinci adımda 23 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 3 değeridir. Ev adresi doludur. Yöntem bir sonraki adrese geçmekte ve boş yer bulunana dek ilerlemektedir. Sırasıyla 3, 4, 5, 6 adresleri dolu bulunmakta, record (kayıt) 7 adresine yazılmaktadır. Bu yerleşim beş probe gerektirmektedir. Bu adımla birlikte toplam probe sayısı 15 olmaktadır. Dolu adreslerin bitişik bir blok oluşturduğu ve her yeni collision (çarpışmanın) bu bloğun tamamını taramak zorunda kaldığı görülmektedir.

ADIM 8 / 9

Progressive overflow: 24

8. anahtar: 24

ev adresi 4 · 5 probe · toplam 20



24 mod 10 = 4

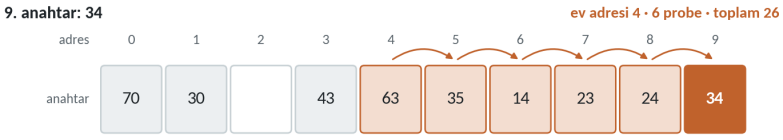
Dolu: 4, 5, 6, 7 → 8 adresine, 5 probe

66

Sekizinci adımda 24 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 4 değeridir. Ev adresi doludur. Yöntem bir sonraki adrese geçmekte ve boş yer bulunana dek ilerlemektedir. Sırasıyla 4, 5, 6, 7 adresleri dolu bulunmakta, record (kayıt) 8 adresine yazılmaktadır. Bu yerleşim beş probe gerektirmektedir. Bu adımla birlikte toplam probe sayısı 20 olmaktadır. Dolu adreslerin bitişik bir blok oluşturduğu ve her yeni collision (çarpışmanın) bu bloğun tamamını taramak zorunda kaldığı görülmektedir.

ADIM 9 / 9

Progressive overflow: 34



34 mod 10 = 4

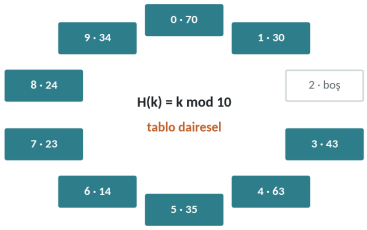
Dolu: 4, 5, 6, 7, 8 → 9 adresine, 6 probe

67

Dokuzuncu adımda 34 anahtarı yerleştirilmektedir. Anahtarın home address (ev adresi) on ile bölümünden kalan olan 4 değeridir. Ev adresi doludur. Yöntem bir sonraki adrese geçmekte ve boş yer bulunana dek ilerlemektedir. Sırasıyla 4, 5, 6, 7, 8 adresleri dolu bulunmakta, record (kayıt) 9 adresine yazılmaktadır. Bu yerleşim altı probe gerektirmektedir. Bu adımla birlikte toplam probe sayısı 26 olmaktadır. Dolu adreslerin bitişik bir blok oluşturduğu ve her yeni collision (çarpışmanın) bu bloğun tamamını taramak zorunda kaldığı görülmektedir.

ÖRNEK

Dokuz anahtarın yerleşimi



Anahtarlar: 35 · 43 · 70 · 63 · 30 · 14 · 23 · 24 · 34

68

Örnekte dokuz anahtar, onluk bölme fonksiyonuyla on gözlü bir tabloya yerleştirilmektedir. İlk üç anahtar home address (ev adreslerine) sorunsuz yerleşir. Dördüncü anahtardan sonra collision (çarpışmalar) başlar ve her yeni anahtar, kendinden önce dolmuş adreslerin üzerinden geçmek zorunda kalır. Son anahtarın yerleşebilmesi için altı adres denenmektedir. Tabloda yalnızca bir göz boş kalmıştır. Toplam yirmi altı probe yapılmakta, ortalama 2,89 olmaktadır. İlk üç anahtarın her biri tek probe ile yerleşirken son üç anahtar birlikte on altı probe tüketmektedir. Bu dağılım, clustering (kümelenmenin) maliyeti son gelen record (kayıtlara) yüklediğini göstermektedir.

ÖRNEK

Adım adım yerleştirme

Anahtar	Ev adresi	Denenen adresler	Yerleşti	Probe
35	5	5	5	1
43	3	3	3	1
70	0	0	0	1
63	3	3 → 4	4	2
30	0	0 → 1	1	2
14	4	4 → 5 → 6	6	3
23	3	3 → 4 → 5 → 6 → 7	7	5
24	4	4 → 5 → 6 → 7 → 8	8	5
34	4	4 → 5 → 6 → 7 → 8 → 9	9	6

Toplam 26 probe · ortalama 2,89: yalnızca 2 numaralı adres boş kaldı

Probe sayıları, yöntemin belirleyici sakıncasını sayısal olarak göstermektedir. İlk üç record (kayıt) birer adımda yerleşirken son kayıtlar beş ve altı adım gerektirmektedir. Dokuz kayıt için ortalama üç adıma yaklaşmaktadır. Sorunun kaynağı kümelenmedir: dolu adresler birbirine bitişik bloklar oluşturur ve her yeni collision (çarpışma) bu bloğun tamamını taramak zorunda kalır. Clustering (kümelenme) büyüdükçe ortalama adım sayısı hızla artar.

DEĞERLENDIRME

Clustering (kümelenme)

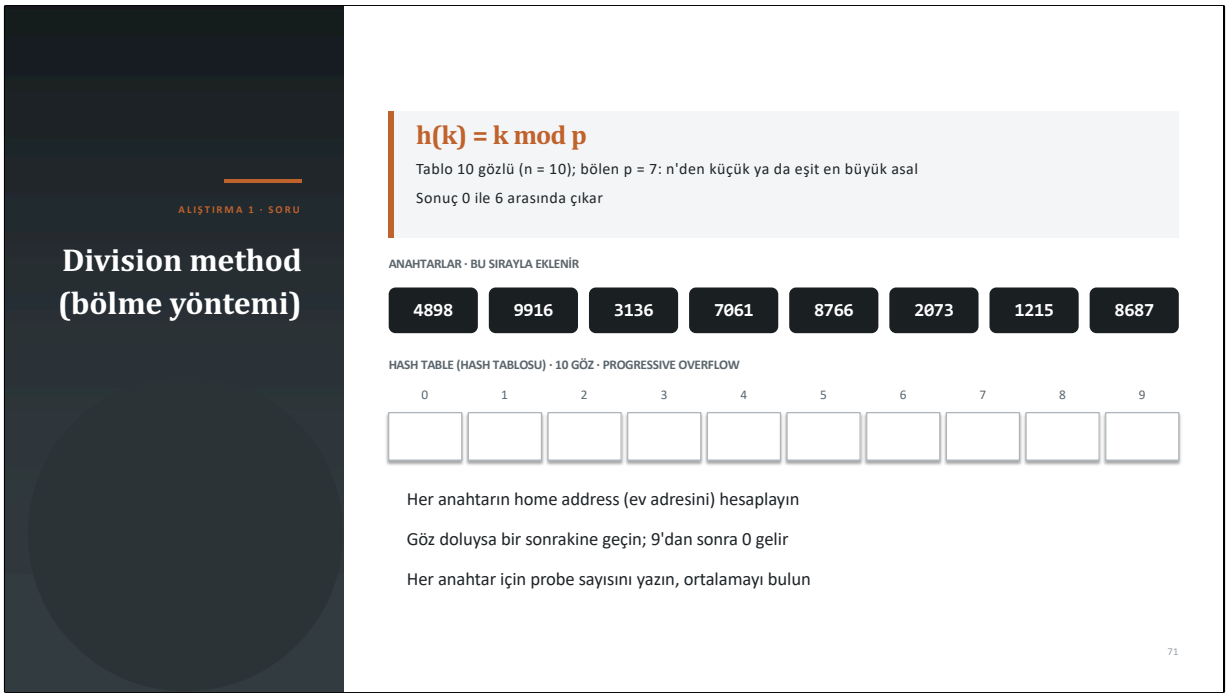
Sorunun kaynağı kümelenmedir

- Dolu adresler birbirine bitişik bloklar oluşturur
- Her yeni collision (çarpışma) bu bloğun tamamını taramak zorunda kalır

Doluluk oranı arttıkça bozulma keskinleşir

İki çıkış yolu vardır

- Variable increment (değişken artım): linear quotient (doğrusal bölüm) yöntemi
- Adres başına record (kayıt) sayısını artırmak: bucket



Hoca'nın Key Mod P tanımı izlenmektedir: n = 10 için bölen, ondan küçük ya da ona eşit en büyük asal olan 7'dir. Öğrencilerden her anahtarın kalanını bulmaları, adresi doluysa progressive overflow (adım adım ilerleyen taşma) ile bir sonraki göze geçmeleri ve her anahtarın probe sayısını yazmaları beklenmektedir. Çözümden önce şu soru tartışılabilir: kalan en fazla 6 olabildiğine göre 7, 8 ve 9 numaralı gözler nasıl dolar?

ALIŞTIRMA 1 · ÇÖZÜM

Division method (bölme yöntemi)

h(k) = k mod p · 10 göz · progressive overflow

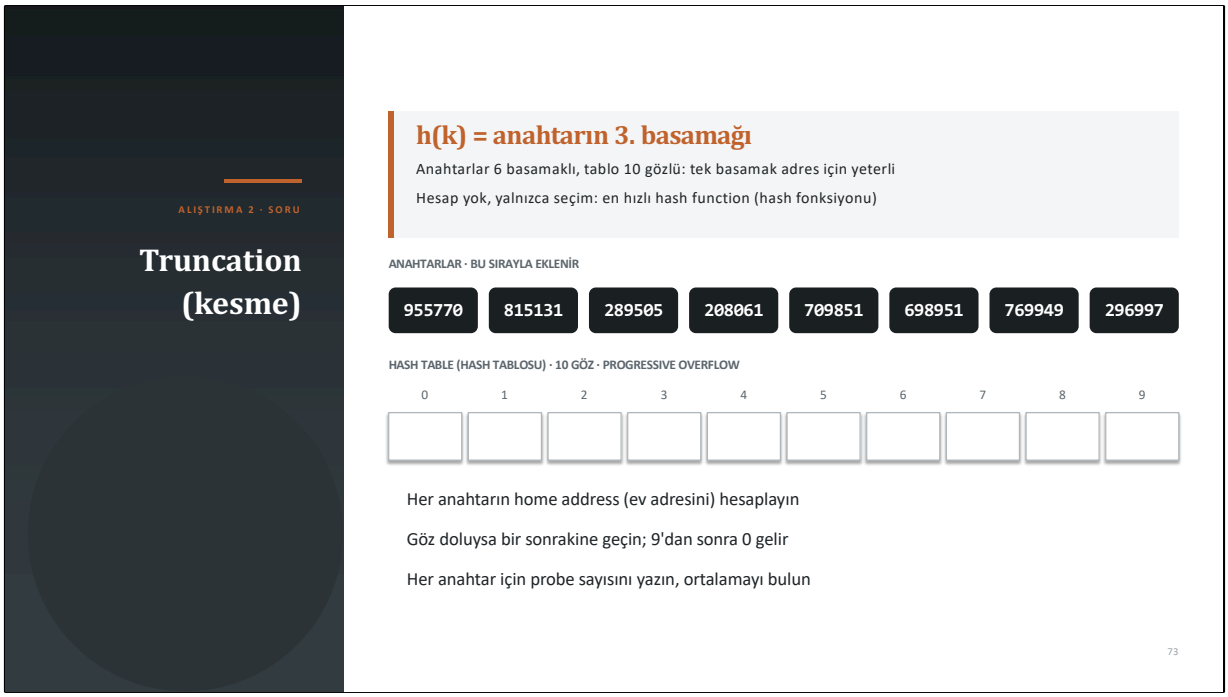
Anahtar	Hesap	Ev adresi	Denenen gözler	Probe
4898	4898 = 7 × 699 + 5	5	5	1
9916	9916 = 7 × 1416 + 4	4	4	1
3136	3136 = 7 × 448 + 0	0	0	1
7061	7061 = 7 × 1008 + 5	5	5→6	2
8766	8766 = 7 × 1252 + 2	2	2	1
2073	2073 = 7 × 296 + 1	1	1	1
1215	1215 = 7 × 173 + 4	4	4→5→6→7	4
8687	8687 = 7 × 1241 + 0	0	0→1→2→3	4

0	1	2	3	4	5	6	7	8	9
3136	2073	8766	8687	9916	4898	7061	1215		

Toplam probe 15 · ortalama 15 / 8 = 1,88

72

Çözüm slaytında her tıklama bir anahtarı getirmektedir: tablodaki satır ve anahtarın hash table (hash tablosundaki) yeri birlikte belirir. Teal renkli gözler anahtarın kendi ev adresine, turuncu gözler taşmayla yerleştiği adrese karşılık gelir. 4898 anahtarının ev adresi 5, göz boş, bir probe. 9916 anahtarının ev adresi 4, göz boş, bir probe. 3136 anahtarının ev adresi 0, göz boş, bir probe. 7061 anahtarının ev adresi 5; 5 dolu olduğundan 6 numaralı göze yerleşir, 2 probe. 8766 anahtarının ev adresi 2, göz boş, bir probe. 2073 anahtarının ev adresi 1, göz boş, bir probe. 1215 anahtarının ev adresi 4; 4, 5, 6 dolu olduğundan 7 numaralı göze yerleşir, 4 probe. 8687 anahtarının ev adresi 0; 0, 1, 2 dolu olduğundan 3 numaralı göze yerleşir, 4 probe. Toplam probe sayısı 15, anahtar başına ortalama 1,88 olmaktadır. Son tıklama toplamı gösterir.



Truncation (kesme) yönteminde anahtarın seçilen basamağı doğrudan adres olarak alınmaktadır. Tablo on gözlü olduğundan tek basamak yeterlidir ve burada üçüncü basamak seçilmiştir. Öğrencilerden adresleri bulmaları, çarpışmaları progressive overflow (adım adım ilerleyen taşma) ile çözmeleri ve probe sayılarını yazmaları beklenmektedir. Seçilen basamakta yalnızca birkaç değerin tekrarlanmasının kümelenmeyi nasıl büyüttüğü bu örnekte gözlenebilir.

ALIŞTIRMA 2 · ÇÖZÜM

Truncation (kesme)

h(k) = anahtarın 3. basamağı · 10 göz · progressive overflow

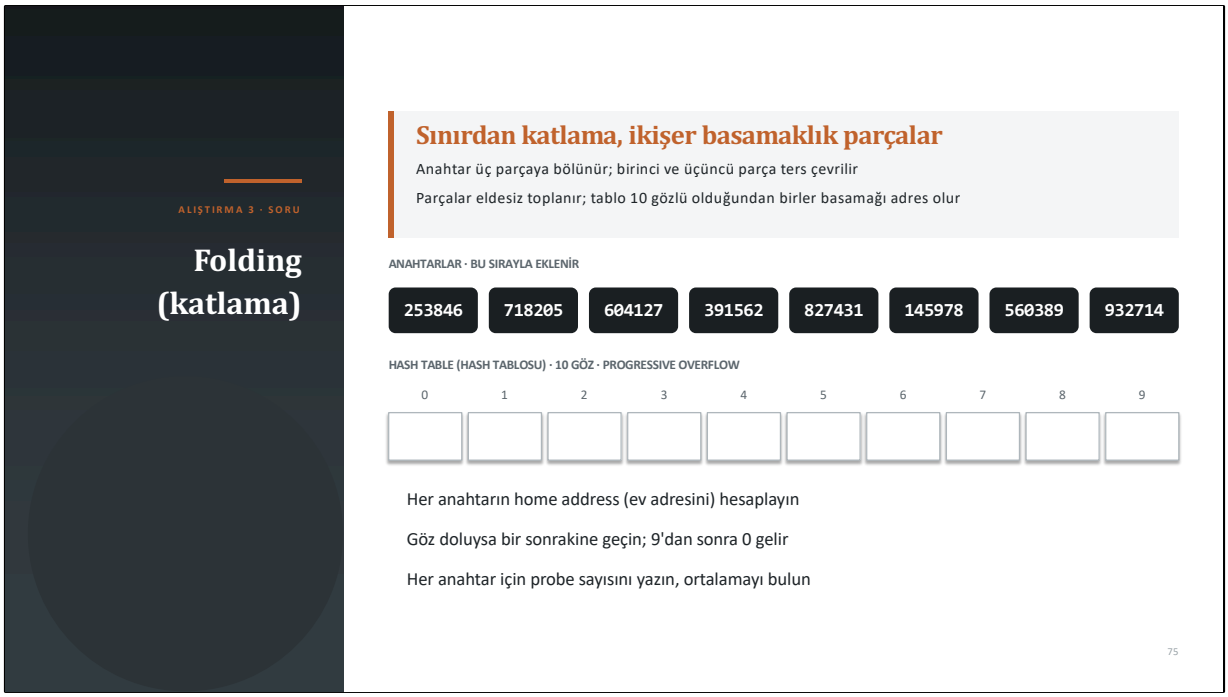
Anahtar	Hesap	Ev adresi	Denenen gözler	Probe
955770	95[5]770	5	5	1
815131	81[5]131	5	5→6	2
289505	28[9]505	9	9	1
208061	20[8]061	8	8	1
709851	70[9]851	9	9→0	2
698951	69[8]951	8	8→9→0→1	4
769949	76[9]949	9	9→0→1→2	4
296997	29[6]997	6	6→7	2

0	1	2	3	4	5	6	7	8	9
709851	698951	769949			955770	815131	296997	208061	289505

Toplam probe 17 · ortalama 17 / 8 = 2,12

74

Çözüm slaytında her tıklama bir anahtarı getirmektedir: tablodaki satır ve anahtarın hash table (hash tablosundaki) yeri birlikte belirir. Teal renkli gözler anahtarın kendi ev adresine, turuncu gözler taşmayla yerleştiği adrese karşılık gelir. 955770 anahtarının ev adresi 5, göz boş, bir probe. 815131 anahtarının ev adresi 5; 5 dolu olduğundan 6 numaralı göze yerleşir, 2 probe. 289505 anahtarının ev adresi 9, göz boş, bir probe. 208061 anahtarının ev adresi 8, göz boş, bir probe. 709851 anahtarının ev adresi 9; 9 dolu olduğundan 0 numaralı göze yerleşir, 2 probe. 698951 anahtarının ev adresi 8; 8, 9, 0 dolu olduğundan 1 numaralı göze yerleşir, 4 probe. 769949 anahtarının ev adresi 9; 9, 0, 1 dolu olduğundan 2 numaralı göze yerleşir, 4 probe. 296997 anahtarının ev adresi 6; 6 dolu olduğundan 7 numaralı göze yerleşir, 2 probe. Toplam probe sayısı 17, anahtar başına ortalama 2,12 olmaktadır. Son tıklama toplamı gösterir.



Hoca'nın sınırdan katlama tanımı ikişer basamaklık parçalarla uygulanmaktadır. 253846 anahtarı 25, 38 ve 46 parçalarına bölünür; dış parçalar ters çevrilerek 52, 38 ve 64 elde edilir. Onlar basamakları 5 + 3 + 6 = 14, birler basamakları 2 + 8 + 4 = 14 olarak eldesiz toplandığında sonuç 44 olur. Tablo on gözlü olduğundan sonucun birler basamağı, yani 4, adres olarak alınır. Ardından progressive overflow (adım adım ilerleyen taşma) uygulanmaktadır.

ALIŞTIRMA 3 · ÇÖZÜM

Folding (katlama)

Sınırdan katlama, ikişer basamaklık parçalar · 10 göz · progressive overflow

Anahtar	Hesap	Ev adresi	Denenen gözler	Probe
253846	52 + 38 + 64 → 44	4	4	1
718205	17 + 82 + 50 → 49	9	9	1
604127	06 + 41 + 72 → 19	9	9→0	2
391562	93 + 15 + 26 → 24	4	4→5	2
827431	28 + 74 + 13 → 05	5	5→6	2
145978	41 + 59 + 87 → 77	7	7	1
560389	65 + 03 + 98 → 56	6	6→7→8	3
932714	39 + 27 + 41 → 97	7	7→8→9→0→1	5

0	1	2	3	4	5	6	7	8	9
604127	932714			253846	391562	827431	145978	560389	718205

Toplam probe 17 · ortalama 17 / 8 = 2,12

76

Çözüm slaytında her tıklama bir anahtarı getirmektedir: tablodaki satır ve anahtarın hash table (hash tablosundaki) yeri birlikte belirir. Teal renkli gözler anahtarın kendi ev adresine, turuncu gözler taşmayla yerleştiği adrese karşılık gelir. 253846 anahtarının ev adresi 4, göz boş, bir probe. 718205 anahtarının ev adresi 9, göz boş, bir probe. 604127 anahtarının ev adresi 9; 9 dolu olduğundan 0 numaralı göze yerleşir, 2 probe. 391562 anahtarının ev adresi 4; 4 dolu olduğundan 5 numaralı göze yerleşir, 2 probe. 827431 anahtarının ev adresi 5; 5 dolu olduğundan 6 numaralı göze yerleşir, 2 probe. 145978 anahtarının ev adresi 7, göz boş, bir probe. 560389 anahtarının ev adresi 6; 6, 7 dolu olduğundan 8 numaralı göze yerleşir, 3 probe. 932714 anahtarının ev adresi 7; 7, 8, 9, 0 dolu olduğundan 1 numaralı göze yerleşir, 5 probe. Toplam probe sayısı 17, anahtar başına ortalama 2,12 olmaktadır. Son tıklama toplamı gösterir.



Mid-square (kare alma) yönteminde anahtarın karesi hesaplanır ve sonucun ortadaki basamakları adres olarak alınır. Üç basamaklı anahtarların kareleri burada beş basamaklıdır; tablo on gözlü olduğundan ortadaki tek basamak, yani üçüncü basamak yeterlidir. Ortadaki basamak anahtarın bütün basamaklarından etkilendiği için birbirine yakın anahtarlar farklı adreslere dağılabilir. Öğrencilerden kareleri hesaplamaları, adresleri bulmaları ve progressive overflow (adım adım ilerleyen taşma) ile yerleştirmeleri beklenmektedir.

ALIŞTIRMA 4 · ÇÖZÜM

Mid-square (kare alma)

Anahtarın karesinin ortadaki basamağı · 10 göz · progressive overflow

Anahtar	Hesap	Ev adresi	Denenen gözler	Probe
286	286² = 81[7]96	7	7	1
311	311² = 96[7]21	7	7→8	2
275	275² = 75[6]25	6	6	1
285	285² = 81[2]25	2	2	1
145	145² = 21[0]25	0	0	1
277	277² = 76[7]29	7	7→8→9	3
104	104² = 10[8]16	8	8→9→0→1	4
251	251² = 63[0]01	0	0→1→2→3	4

0	1	2	3	4	5	6	7	8	9
145	104	285	251			275	286	311	277

Toplam probe 17 · ortalama 17 / 8 = 2,12

78

Çözüm slaytında her tıklama bir anahtarı getirmektedir: tablodaki satır ve anahtarın hash table (hash tablosundaki) yeri birlikte belirir. Teal renkli gözler anahtarın kendi ev adresine, turuncu gözler taşmayla yerleştiği adrese karşılık gelir. 286 anahtarının ev adresi 7, göz boş, bir probe. 311 anahtarının ev adresi 7; 7 dolu olduğundan 8 numaralı göze yerleşir, 2 probe. 275 anahtarının ev adresi 6, göz boş, bir probe. 285 anahtarının ev adresi 2, göz boş, bir probe. 145 anahtarının ev adresi 0, göz boş, bir probe. 277 anahtarının ev adresi 7; 7, 8 dolu olduğundan 9 numaralı göze yerleşir, 3 probe. 104 anahtarının ev adresi 8; 8, 9, 0 dolu olduğundan 1 numaralı göze yerleşir, 4 probe. 251 anahtarının ev adresi 0; 0, 1, 2 dolu olduğundan 3 numaralı göze yerleşir, 4 probe. Toplam probe sayısı 17, anahtar başına ortalama 2,12 olmaktadır. Son tıklama toplamı gösterir.

KAVRAM

Bucket



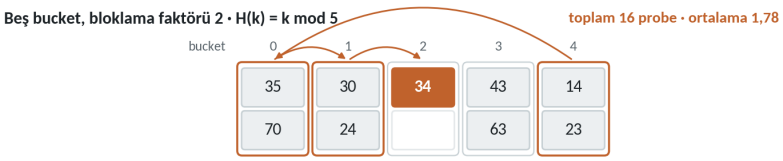
Aynı adres bölgesine düşen record (kayıtlar) collision (çarpışma) sayılmaz

Bir bucket'taki kayıt sayısına blocking factor (bloklama faktörü) denir

Bir record (kayıt) alanına birden çok kaydın yazılmasına bucket denir. Aynı adres bölgesine düşen kayıtlar aynı bucket içinde yer aldığında collision (çarpışma) sayılmaz ve erişim tek okumayla yapılır. Bir bucket içinde saklanabilen kayıt sayısına blocking factor (bloklama faktörü) denmektedir.

ÖRNEK

Aynı anahtarlar, bucket'la



Aynı on yuva: probe 26'dan 16'ya iner

Son anahtar 34: dört bucket dener

Aynı dokuz anahtar, bu kez beş bucket'tan oluşan ve her bucket'ında iki record (kayıt) tutan bir tabloya yerleştirilmektedir. Toplam yuva sayısı yine ondur. Home address (ev adresi) anahtarın beşe bölümünden kalandır. İlk dört anahtarın hepsi, aynı adrese düşseler bile, ev bucket'larındaki boş yuvaya yerleşmektedir. Overflow (taşma) ancak bir bucket'ın iki yuvası dolduğunda başlamaktadır. Otuz anahtarı dolu sıfırıncı bucket'tan birinciye, yirmi üç anahtarı üçüncüden dördüncüye taşmaktadır. Son anahtar 34 dördüncü, sıfırıncı ve birinci bucket'ları dolu bulmakta ve ikinci bucket'a yerleşmektedir. Toplam probe sayısı 16, ortalama 1,78 olmaktadır. Progressive overflow ile aynı yerde yirmi altı probe gerektiği hatırlanırsa, bir probe'un bir blok okuması olduğu dosya ortamında kazanç direct access (doğrudan erişim) sayısına yansımaktadır.

7

Linear quotient (doğrusal bölüm)

Variable increment (değişken artımlı) collision resolution (çarpışma çözümleme)

Bu bölümde bir önceki bölümde tanıtılan progressive overflow (adım adım ilerleyen taşmanın) geliştirilmiş bir biçimi ele alınmaktadır. Yöntemin belirleyici sakıncası clustering (kümelenmeydi). Dolu adreslerin bitişik bloklar oluşturması ortalama probe sayısını yükseltiyordu. Bu bölümdeki yöntem, sabit artım yerine anahtardan hesaplanan bir artım kullanarak kümelenmeyi dağıtmaktadır.

HATIRLATMA

Clustering (kümelenme) sorunu

Progressive overflow (adım adım ilerleyen taşmada) artım her zaman birdir

Dolu adresler bitişik öbekler oluşturur

- Öbeğe düşen her yeni anahtar öbeğin tamamını tarar
- Tarayan record (kayıt) öbeğin sonuna eklenir: öbek büyür

Önceki bölümdeki örnekte ortalama probe sayısı üçe yaklaşmıştı

Progressive overflow (adım adım ilerleyen taşmada) çarpışan record (kayıt) bir sonraki adrese, o da doluysa ondan sonrakine yerleştirilir. Artım her zaman birdir. Sonuç, dolu adreslerin bitişik öbekler oluşturmasıdır. Bu öbeğe düşen her yeni anahtar öbeğin tamamını taramak zorunda kalır ve öbek büyür. Önceki bölümdeki örnekte dokuz anahtar için ortalama probe sayısı üçe yaklaşmıştı.

FIKIR

Variable increment (değişken artım)

Yöntem progressive overflow (adım adım ilerleyen taşmaya) oldukça benzer

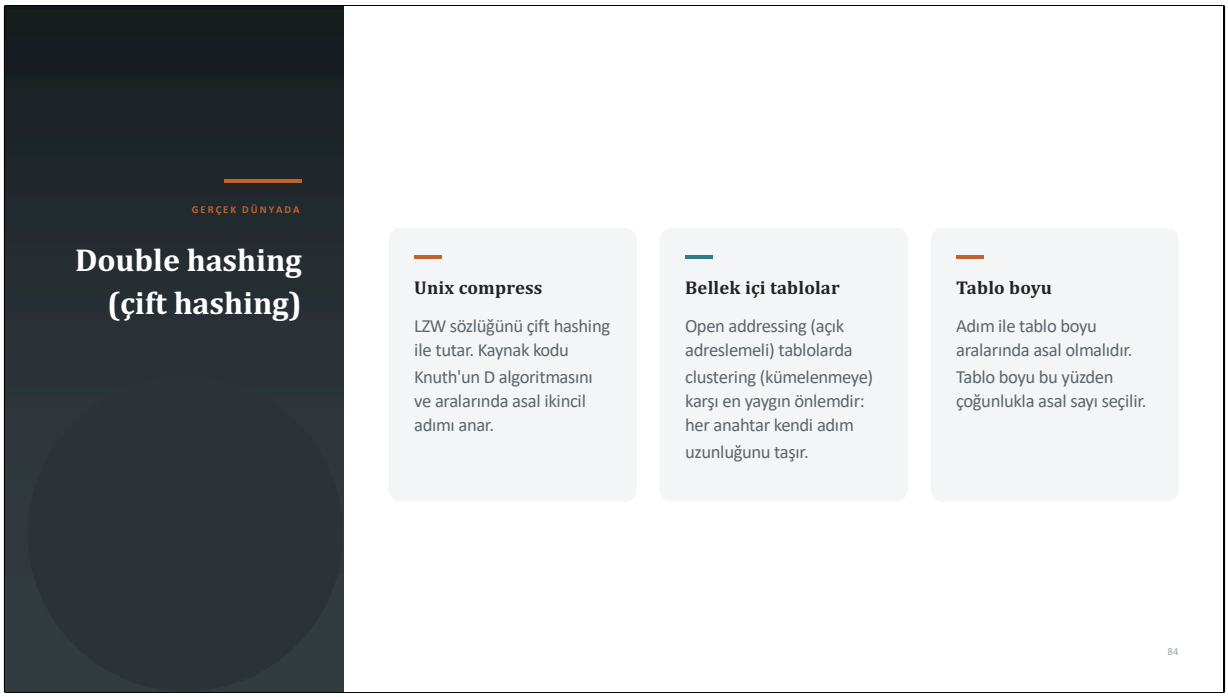
Tek fark: bir artım yerine değişken artım kullanılır

Artımı ikinci bir hash function (hash fonksiyonu) belirler

- Collision (çarpışmada) record (kaydın) kaç sıra sonra yerleşeceğini bulur

İki hash fonksiyonu kullanıldığı için çift hash ailesindendir

Linear quotient (doğrusal bölüm) yöntemi, progressive overflow (adım adım ilerleyen taşmaya) oldukça benzer. Aralarındaki tek fark, record (kayıt) eklemede bir artım yerine variable increment (değişken artım) kullanılmasıdır. Collision (çarpışma) durumunda kaydın kaç sıra sonra yerleştirileceğini belirleyen ikinci bir hash function (hash fonksiyonu) kullanılır. İki ayrı hash fonksiyonu kullanıldığı için yöntem çift hash algoritmaları ailesine aittir. İkinci fonksiyonun görevi, aynı home address (ev adresine) düşen anahtarlara farklı artımlar vererek her birini farklı bir yola göndermektir. Birincil clustering (kümelenme) böylece dağılır.



Linear quotient (doğrusal bölüm), literatürde double hashing (çift hashing) adıyla bilinen yöntemin bir biçimidir. Unix ortamındaki compress programı, Lempel-Ziv-Welch sözlüğünü çift hashing ile tutar; kaynak kodundaki açıklamada Knuth'un D algoritması ve tablo boyuyla aralarında asal ikincil adım açıkça anılmaktadır. Compression (sıkıştırma) altıncı haftada ele alındığında bu bağlantıya dönülecektir. Open addressing (açık adreslemeli) tablolarda clustering (kümelenmeye) karşı en yaygın önlem budur: aynı home address (ev adresine) düşen anahtarlar farklı adım uzunlukları aldığından farklı yönlere dağılır. Adımın tablo boyuyla aralarında asal olması gerektiğinden tablo boyu çoğunlukla asal sayı seçilir.

ALGORITMA

Record (kayıt) ekleme

Anahtara birinci hash function (hash fonksiyonunu) uygula: home address (ev adresi)

Ev adresi boşsa kaydı oraya ekle

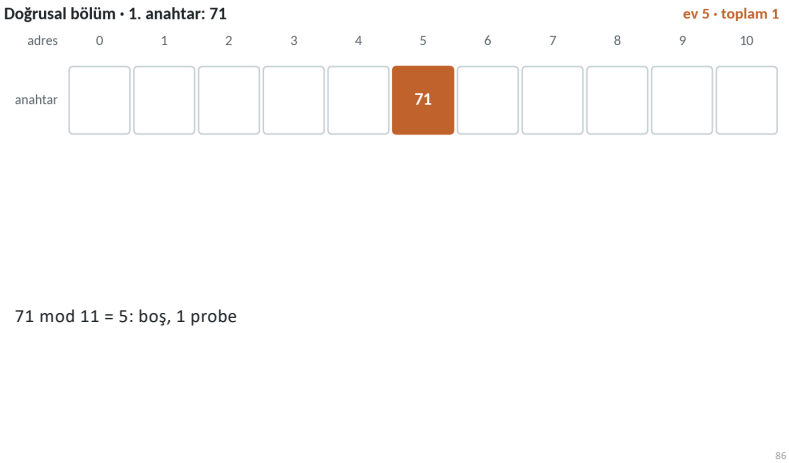
Değilse

- İkinci fonksiyondan artımı hesapla
- Sonuç sıfırsa artımı bire eşitle
- Çift kayıt durumunu sına
- Adresi artım kadar ilerlet, tablo dairesel düşünülür
- Boş adres bulunana dek sürdür

Ekleme algoritması şöyle yürür. Önce anahtara birinci hash function (hash fonksiyonu) uygulanarak home address (ev adresi) bulunur. Ev adresi boşsa record (kayıt) oraya yazılır. Doluysa ikinci hash fonksiyonundan artım hesaplanır. Artımın sıfır çıkması durumunda bir alınır. Adres bu artım kadar ilerletilir ve tablo dairesel düşünülerek boş yer aranır. Çift kayıt durumu ayrıca sınanır. Boş adres bulunduğunda kayıt oraya yazılır.

ADIM 1 / 9

Linear quotient (doğrusal bölüm): 71

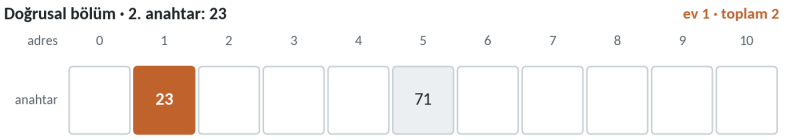


71 mod 11 = 5: boş, 1 probe

Birinci adımda 71 anahtarı eklenmektedir. Home address (ev adresi) boş olduğundan record (kayıt) tek probe ile yerleşmektedir ve ikinci hash function (hash fonksiyonuna) başvurulmamaktadır. Linear quotient (doğrusal bölüm) ile progressive overflow (adım adım ilerleyen taşma), collision (çarpışma) olmadığı sürece aynı biçimde davranmaktadır. Toplam probe sayısı bu adımdan sonra 1 olmaktadır.

ADIM 2 / 9

Linear quotient (doğrusal bölüm): 23



23 mod 11 = 1: boş, 1 probe

İkinci adımda 23 anahtarı eklenmektedir. Home address (ev adresi) boş olduğundan record (kayıt) tek probe ile yerleşmektedir ve ikinci hash function (hash fonksiyonuna) başvurulmamaktadır. Linear quotient (doğrusal bölüm) ile progressive overflow (adım adım ilerleyen taşma), collision (çarpışma) olmadığı sürece aynı biçimde davranmaktadır. Toplam probe sayısı bu adımdan sonra 2 olmaktadır.

ADIM 3 / 9

Linear quotient (doğrusal bölüm): 73



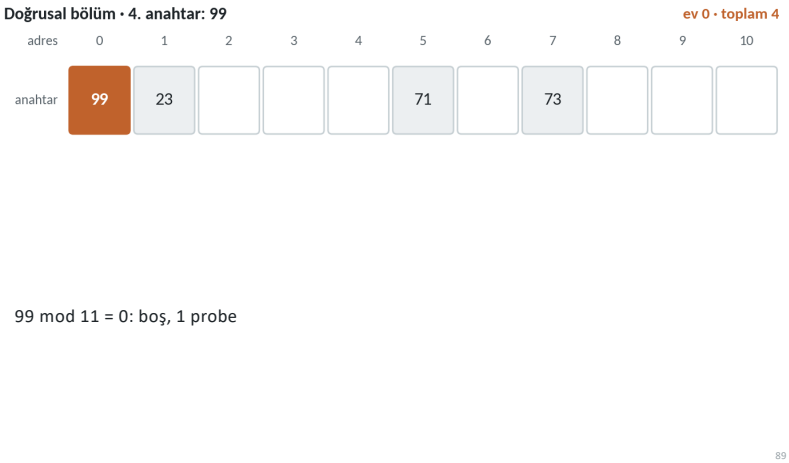
73 mod 11 = 7: boş, 1 probe

88

Üçüncü adımda 73 anahtarı eklenmektedir. Home address (ev adresi) boş olduğundan record (kayıt) tek probe ile yerleşmektedir ve ikinci hash function (hash fonksiyonuna) başvurulmamaktadır. Linear quotient (doğrusal bölüm) ile progressive overflow (adım adım ilerleyen taşma), collision (çarpışma) olmadığı sürece aynı biçimde davranmaktadır. Toplam probe sayısı bu adımdan sonra 3 olmaktadır.

ADIM 4 / 9

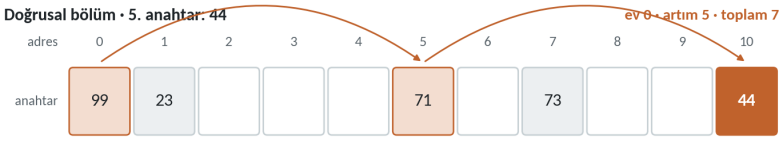
Linear quotient (doğrusal bölüm): 99



Dörtüncü adımda 99 anahtarı eklenmektedir. Home address (ev adresi) boş olduğundan record (kayıt) tek probe ile yerleşmektedir ve ikinci hash function (hash fonksiyonuna) başvurulmamaktadır. Linear quotient (doğrusal bölüm) ile progressive overflow (adım adım ilerleyen taşma), collision (çarpışma) olmadığı sürece aynı biçimde davranmaktadır. Toplam probe sayısı bu adımdan sonra 4 olmaktadır.

ADIM 5 / 9

Linear quotient (doğrusal bölüm): 44



44 mod 11 = 0 dolu · artım 7 − (44 mod 7) = 5

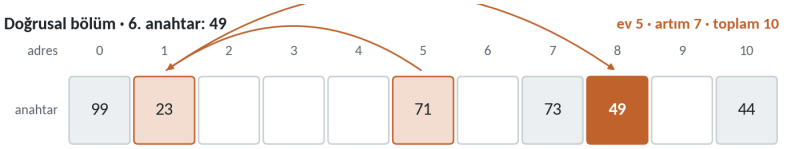
Yol 0 → 5 → 10: 3 probe

90

Beşinci adımda 44 anahtarı eklenmektedir. Home address (ev adresi) doludur. İkinci hash function (hash fonksiyonu) artımı yedi eksi 44 sayısının yediye bölümünden kalanı, yani 5 olarak vermektedir. Adres her adımda 5 artırılmakta ve on bire göre dairesel olarak ilerlenmektedir. Denenen adresler sırasıyla 0 → 5 → 10 olup record (kayıt) 10 adresine yerleşmektedir. Toplam probe sayısı bu adımdan sonra 7 olmaktadır.

ADIM 6 / 9

Linear quotient (doğrusal bölüm): 49



49 mod 11 = 5 dolu · artım 7 − (49 mod 7) = 7

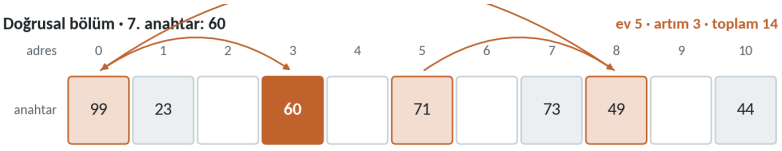
Yol 5 → 1 → 8: 3 probe

91

Altıncı adımda 49 anahtarı eklenmektedir. Home address (ev adresi) doludur. İkinci hash function (hash fonksiyonu) artımı yedi eksi 49 sayısının yediye bölümünden kalanı, yani 7 olarak vermektedir. Adres her adımda 7 artırılmakta ve on bire göre dairesel olarak ilerlenmektedir. Denenen adresler sırasıyla 5 → 1 → 8 olup record (kayıt) 8 adresine yerleşmektedir. Aynı ev adresine düşen anahtarların farklı artımlar alması, bitişik öbek oluşmasını engellemektedir. Toplam probe sayısı bu adımdan sonra 10 olmaktadır.

ADIM 7 / 9

Linear quotient (doğrusal bölüm): 60



60 mod 11 = 5 dolu · artım 7 − (60 mod 7) = 3

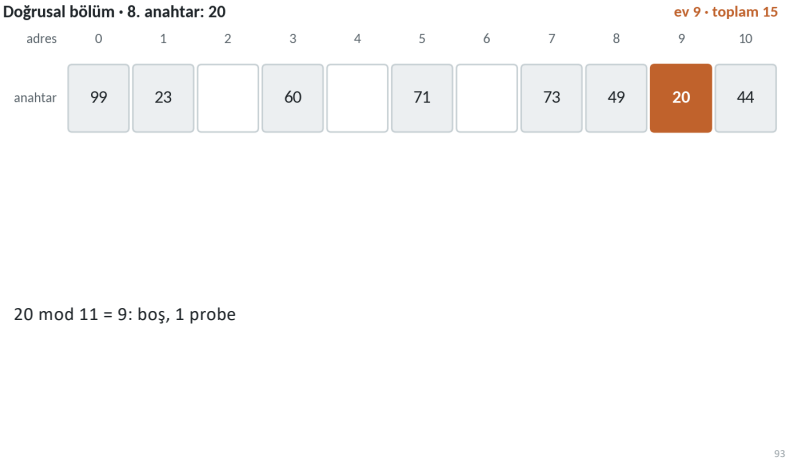
Yol 5 → 8 → 0 → 3: 4 probe

92

Yedinci adımda 60 anahtarı eklenmektedir. Home address (ev adresi) doludur. İkinci hash function (hash fonksiyonu) artımı yedi eksi 60 sayısının yediye bölümünden kalanı, yani 3 olarak vermektedir. Adres her adımda 3 artırılmakta ve on bire göre dairesel olarak ilerlenmektedir. Denenen adresler sırasıyla 5 → 8 → 0 → 3 olup record (kayıt) 3 adresine yerleşmektedir. Aynı ev adresine düşen anahtarların farklı artımlar alması, bitişik öbek oluşmasını engellemektedir. Toplam probe sayısı bu adımdan sonra 14 olmaktadır.

ADIM 8 / 9

Linear quotient (doğrusal bölüm): 20

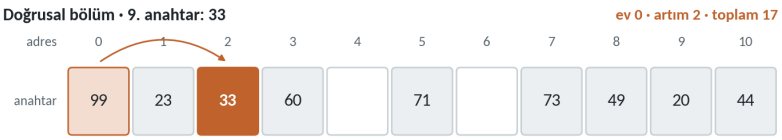


93

Sekizinci adımda 20 anahtarı eklenmektedir. Home address (ev adresi) boş olduğundan record (kayıt) tek probe ile yerleşmektedir ve ikinci hash function (hash fonksiyonuna) başvurulmamaktadır. Linear quotient (doğrusal bölüm) ile progressive overflow (adım adım ilerleyen taşma), collision (çarpışma) olmadığı sürece aynı biçimde davranmaktadır. Toplam probe sayısı bu adımdan sonra 15 olmaktadır.

ADIM 9 / 9

Linear quotient (doğrusal bölüm): 33



33 mod 11 = 0 dolu · artım 7 − (33 mod 7) = 2

Yol 0 → 2: 2 probe

94

Dokuzuncu adımda 33 anahtarı eklenmektedir. Home address (ev adresi) doludur. İkinci hash function (hash fonksiyonu) artımı yedi eksi 33 sayısının yediye bölümünden kalanı, yani 2 olarak vermektedir. Adres her adımda 2 artırılmakta ve on bire göre dairesel olarak ilerlenmektedir. Denenen adresler sırasıyla 0 → 2 olup record (kayıt) 2 adresine yerleşmektedir. Toplam probe sayısı bu adımdan sonra 17 olmaktadır.

ÖZET

Adım adım yerleştirme

$H_1(k) = k \bmod 11$ $H_2(k) = 7 - (k \bmod 7)$

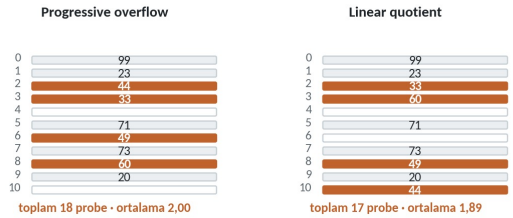
Anahtar	Ev adresi	Artım	Denenen adresler	Yerleşti	Probe
71	5	:	5	5	1
23	1	:	1	1	1
73	7	:	7	7	1
99	0	:	0	0	1
44	0	5	0 → 5 → 10	10	3
49	5	7	5 → 1 → 8	8	3
60	5	3	5 → 8 → 0 → 3	3	4
20	9	:	9	9	1
33	0	2	0 → 2	2	2

Toplam 17 probe · ortalama 1,89: aynı home address (ev adresine) düşen anahtarlar farklı yol izliyor

Tabloda dokuz anahtarın yerleşimi izlenmektedir. Birinci fonksiyon on bir bölenli bölme, ikinci fonksiyon yediden anahtarın yedi bölümünden kalanının çıkarılmasıdır. İlk dört anahtar home address (ev adreslerine) yerleşir. Çarpışan anahtarlar için artım her anahtarda farklıdır. Bu nedenle aynı ev adresine düşen iki anahtar farklı yollar izler. Toplam probe sayısı on yedi, ortalama 1,89 çıkmaktadır.

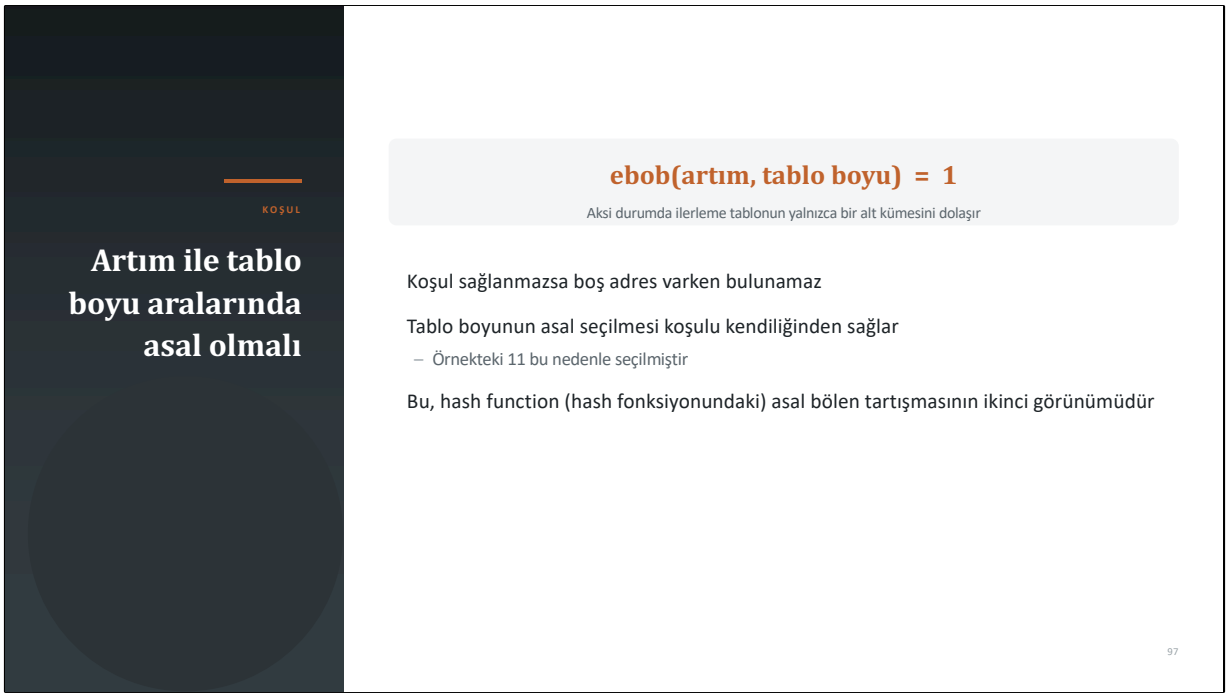
KARŞILAŞTIRMA

Aynı anahtarlar, iki yöntem

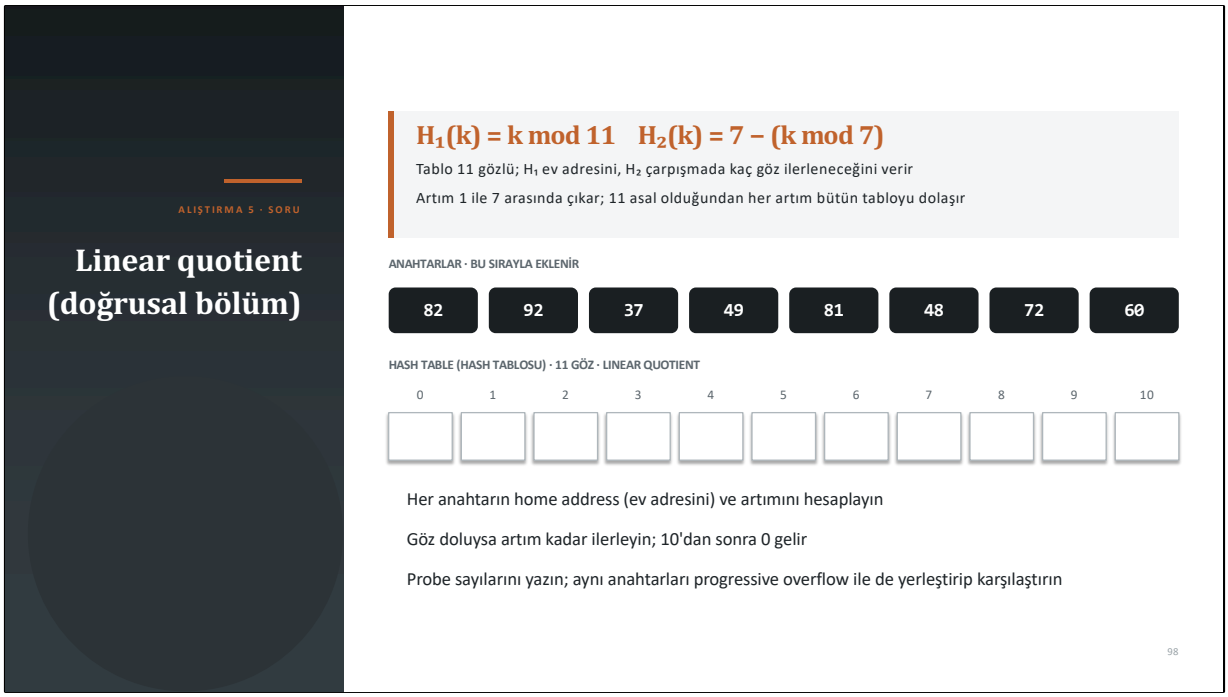


Sayısal fark küçük, asıl değişiklik taşan record (kayıtların) dağılımında

Aynı anahtar kümesi progressive overflow (adım adım ilerleyen taşmayla) yerleştirildiğinde toplam probe sayısı on sekiz, ortalama 2,00 çıkmaktadır. Sayısal fark küçüktür, ancak yerleşimlere bakıldığında asıl değişiklik görülür: birinci yöntemde taşan record (kayıtlar) sıfır ile üç arasındaki adreslere bitişik olarak yığılırken, ikinci yöntemde tabloya dağılmaktadır. Dağılım, doluluk oranı yükseldikçe belirleyici duruma gelir. Adım adım ilerleyen taşmada kırk dört, kırk dokuz ve altmış anahtarları sıfırıncı ve beşinci adreslerden başlayan öbeklere eklenmektedir. Linear quotient (doğrusal bölümde) aynı anahtarlar onuncu, sekizinci ve üçüncü adreslere dağılmaktadır.



Variable increment (değişken artımın) işe yaraması için bir koşul vardır: artım ile tablo boyunun aralarında asal olması gerekir. Aksi durumda ilerleme tablonun tamamını dolaşmaz, yalnızca belirli adreslerden oluşan bir alt kümeyi dolaşır ve boş adres bulunsa bile bulunamaz. Tablo boyunun asal seçilmesi bu koşulu kendiliğinden sağlar. Örnekteki on bir bu nedenle seçilmiştir.



Bu alıştırmada linear quotient (doğrusal bölüm) yöntemi, anlatımdaki fonksiyonlarla ancak yeni bir anahtar kümesiyle uygulanmaktadır. Birinci hash function (hash fonksiyonu) on bire bölümden kalanı, ikinci fonksiyon ise yediden anahtarın yediye bölümünden kalanının çıkarılmasını vermektedir. Öğrencilerden her anahtarın ev adresini ve artımını bulmaları, adres doluysa artım kadar ilerleyerek boş göz aramaları ve probe sayılarını yazmaları beklenmektedir. Sekiz anahtarın beşinin ev adresi dört ya da beştir; aynı kümenin progressive overflow (adım adım ilerleyen taşma) ile de yerleştirilmesi, iki yöntem arasındaki farkın sayısal olarak görülmesini sağlar.

ALIŞTIRMA 5 · ÇÖZÜM

Linear quotient (doğrusal bölüm)

$H_1(k) = k \bmod 11$ · $H_2(k) = 7 - (k \bmod 7)$ · 11 göz

Anahtar	H_1 (ev adresi)	H_2 (artım)	Denenen gözler	Probe
82	5	–	5	1
92	4	–	4	1
37	4	5	4→9	2
49	5	7	5→1	2
81	4	3	4→7	2
48	4	1	4→5→6	3
72	6	5	6→0	2
60	5	3	5→8	2

0	1	2	3	4	5	6	7	8	9	10
72	49			92	82	48	81	60	37	

Toplam probe 15 · ortalama 1,88 progressive overflow ile: 31 probe, ortalama 3,88

99

Çözüm slaytında her tıklama bir anahtarı getirmektedir. 82 anahtarı 5 numaralı ev adresine yerleşir. 92 anahtarı 4 numaralı ev adresine yerleşir. 37 anahtarının ev adresi 4 doludur; artım 5 ile 9 numaralı göze 2 probe ile yerleşir. 49 anahtarının ev adresi 5 doludur; artım 7 ile 1 numaralı göze 2 probe ile yerleşir. 81 anahtarının ev adresi 4 doludur; artım 3 ile 7 numaralı göze 2 probe ile yerleşir. 48 anahtarının ev adresi 4 doludur; artım 1 ile 6 numaralı göze 3 probe ile yerleşir. 72 anahtarının ev adresi 6 doludur; artım 5 ile 0 numaralı göze 2 probe ile yerleşir. 60 anahtarının ev adresi 5 doludur; artım 3 ile 8 numaralı göze 2 probe ile yerleşir. Toplam probe sayısı 15, ortalama 1,88 olmaktadır. Aynı anahtarlar progressive overflow (adım adım ilerleyen taşma) ile yerleştirildiğinde dört, beş ve altı numaralı gözlerin çevresinde bir küme oluşmakta ve toplam 31 probe yapılmaktadır. 48 anahtarının artımı bir çıktığından bu anahtar progressive overflow ile aynı yolu izlemektedir; değişken artımın kazancı, artımları farklı olan anahtarların kümeye girmeden dağılmasından gelmektedir.

8

Uygulama

Algoritmaların C++ gerçeklemesi

Bu bölümde algoritma hattındaki yöntemler C++ ile gerçeklenmektedir. Önce load factor (depolama oranının) collision (çarpışma) sayısına etkisi bölme yöntemiyle ölçülmekte, ardından progressive overflow (adım adım ilerleyen taşma) ve linear quotient (doğrusal bölüm) yöntemlerinin ekleme fonksiyonları verilmektedir. Kodun ürettiği probe sayıları, bölüm içinde elle izlenen örneklerle karşılaştırılmaktadır. Elle çözülen sınıf alıştırmaları ilgili algoritmaların anlatımının hemen ardına alınmıştır.

C++ · 1

Division method (bölme yöntemiyle) collision (çarpışma) sayımı

```
const int anahtarlar[] = {71, 23, 73, 99, 44, 49, 60, 20, 33};
for (int n : {11, 13, 17, 23}) {
    vector<int> sayac(n, 0);
    int carpisma = 0;
    for (int k : anahtarlar)
        if (sayac[k % n]++ > 0) ++carpisma;
    int kullanilan = count_if(sayac.begin(), sayac.end(),
                              [](int s) { return s > 0; });
    cout << "n=" << n << "  kullanilan=" << kullanilan
         << "  carpisma=" << carpisma << '\n';
}
```

ÇIKTI

```
n=11  kullanilan=5  carpisma=4
n=13  kullanilan=5  carpisma=4
n=17  kullanilan=8  carpisma=1
n=23  kullanilan=9  carpisma=0
```

C++ · 2

Progressive overflow (adım adım ilerleyen taşma)

```
constexpr int BOS = -1;

int ekle(vector<int>& tablo, int anahtar, int& probe) {
    const int n = static_cast<int>(tablo.size());
    const int ev = anahtar % n;
    int adres = ev;
    probe = 1;

    while (tablo[adres] != BOS) {
        adres = (adres + 1) % n;
        ++probe;
        if (adres == ev) return -1;
    }
    tablo[adres] = anahtar;
    return adres;
}
```

ÇÖZÜMLÜ ÖRNEK

Dokuz anahtar, on göz

```
vector<int> tablo(10, BOS);
int toplam = 0, probe, i = 0;
for (int k : {35, 43, 70, 63, 30, 14, 23, 24, 34}) {
    int a = ekle(tablo, k, probe);
    toplam += probe;
    cout << k << "->" << a << " (" << probe << ")"
         << (++i % 3 ? "   " : "\n");
}
cout << "toplam " << toplam << ", ortalama " << toplam / 9.0;
```

ÇIKTI

```
35->5 (1)   43->3 (1)   70->0 (1)
63->4 (2)   30->1 (2)   14->6 (3)
23->7 (5)   24->8 (5)   34->9 (6)
toplam 26, ortalama 2.88889
```

26 probe

Parantez içi: o anahtarın probe sayısı

C++ · 3

Linear quotient (doğrusal bölüm)

```
int ekle(vector<int>& tablo, int anahtar, int& probe) {
    const int n  = static_cast<int>(tablo.size());
    const int ev = anahtar % n;
    int artim    = 7 - (anahtar % 7);
    if (artim == 0) artim = 1;

    int adres = ev;
    probe = 1;
    while (tablo[adres] != BOS) {
        adres = (adres + artim) % n;
        ++probe;
        if (adres == ev) return -1;
    }
    tablo[adres] = anahtar;
    return adres;
}
```

Progressive overflow (adım adım ilerleyen taşmanın) kodundan tek satırla ayrılır: artımın hesaplanması

ÇÖZÜMLÜ ÖRNEK

Linear quotient (doğrusal bölüm), dokuz anahtar

```
vector<int> tablo(11, BOS);
int toplam = 0, probe, i = 0;
for (int k : {71, 23, 73, 99, 44, 49, 60, 20, 33}) {
    int a = ekle(tablo, k, probe);
    toplam += probe;
    cout << k << "->" << a << " (" << probe << ")"
         << (++i % 3 ? "   " : "\n");
}
cout << "toplam " << toplam << ", ortalama " << toplam / 9.0;
```

ÇIKTI

```
71->5 (1)   23->1 (1)   73->7 (1)
99->0 (1)   44->10 (3)   49->8 (3)
60->3 (4)   20->9 (1)   33->2 (2)
toplam 17, ortalama 1.88889
```

17 probe
44, 49, 60 farklı artımlarla dağılıyor